ALPHA MICROSYSTEMS
RIGHT. FROM THE START.

AlphaWRITE[â] Programmer's Information Manual

DSM-00076-02

© 1995 Alpha Microsystems

REVISIONS INCORPORATED	
REVISION	DATE
00	March 1990
01	April 1991
02	May 1992

AlphaWRITE Programmer's Information Manual

To re-order this document, request part number DSO-00076-00.

This document applies to AlphaWRITE 2.1A and later

The information contained in this manual is believed to be accurate and reliable. However, no responsibility for the accuracy, completeness or use of this information is assumed by Alpha Microsystems.

This document may contain references to products covered under U.S. Patent Number 4,530,048.

The following are registered trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AMIGOS
AMOS
Alpha Micro
AlphaACCOUNTING
AlphaBASIC
AlphaCALC
AlphaCOBOL
AlphaDDE
AlphaFORTRAN 77
AlphaLAN
AlphaLEDGER
AlphaMAIL
AlphaMATE
AlphaNET
AlphaPASCAL
AlphaRJE
AlphaWRITE
CASELODE
OmniBASIC
VER-A-TEL
VIDEOTRAX

The following are trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AlphaBASIC PLUS
AlphaVUE
AM-PC
AMTEC
DART
ESP
MULTI
*in*Sight/am
*in*Front/am

All other copyrights and trademarks are the property of their respective holders.

ALPHA MICROSYSTEMS
2722 S. Fairview St.
P.O. Box 25059
Santa Ana, CA 92799

TABLE OF CONTENTS

PRODUCTION NOTES.....vii

CHAPTER ONE - INTRODUCTION TO AlphaWRITE PROGRAMMER'S INFORMATION
HOW THIS BOOK IS ORGANIZED.....1-1
CONVENTIONS USED IN THIS MANUAL.....1-1
THE AlphaWRITE DOCUMENTATION LIBRARY..... 1-3

CHAPTER TWO - EXTENDED PRINTER DRIVER SPECIFICATIONS..... 2-1
2.1 PRINTER DRIVER FUNDAMENTALS..... 2-1
2.1.1 CREATING A NEW PRINTER DRIVER..... 2-1
2.1.2 PDVSYM.M68.....2-2
2.2 PRINTER DRIVER STRUCTURE..... 2-2
2.2.1 ENTRY POINTS..... 2-2
2.2.2 PROGRAM HEADER..... 2-4
2.2.3 CAPABILITY FLAGS..... 2-4
2.2.4 EXTENDED PRINTER DRIVERS..... 2-5
2.2.4.1 Compatibility.....2-6
2.2.4.2 Implementation Considerations.....2-6
2.2.4.3 Recommendation..... 2-6
2.2.5 VERSION INTERACTION..... 2-6
2.2.6 PRINTER DRIVERS AND YOUR APPLICATION.....2-7
2.2.6.1 Division Of Responsibility.....2-7
2.2.6.2 The Driver's Communication Area..... 2-8
2.2.6.3 The Driver's Impure Area..... 2-8
2.2.6.4 Using A Printer Driver In An Application..... 2-8
2.2.6.5 Register Calling Conventions.....2-9
2.2.6.6 Driver Initialization.....2-10
2.2.6.7 Outputting Characters.....2-10
2.2.6.8 Outputting New Lines.....2-10
2.2.6.9 Outputting New Pages.....2-10
2.2.6.10 Finishing Up.....2-11
2.2.6.11 Examples Of Using .PDV Calls.....2-11
A Basic Implementation.....2-11
Refinement #1 - Simple Right Justification.....2-12
Refinement #2 - Adding Proportional Spacing Support.....2-13
Refinement #3 - Adding Support for Proportional Fonts.....2-14
Refinement #4 - Adding Track Kerning.....2-15
2.3 CAPABILITY FLAGS..... 2-15
2.3.1 PD$VAR - VARIABLE SPACING.....2-15
2.3.2 PD$PRO - PROPORTIONAL MOVEMENT.....2-16
2.3.3 PD$FED - CUT SHEET FEEDER..... 2-16
2.3.4 PD$EXT - EXTENDED PRINTER DRIVER.....2-16
2.3.5 PD$PTS - PRINTER DRIVER IS POINTS-ORIENTED.....2-16
2.3.6 PD$SAV - PRINTER CAN SAVE/RESTORE PRINT STATE.....2-16

2.3.7 PD$BLD - PRINTER SUPPORTS BOLD..... 2-17
2.3.8 PD$UND - PRINTER SUPPORTS UNDERSCORE.....2-17
2.3.9 PD$ITL - PRINTER SUPPORTS ITALICS.....2-17
2.3.10 PD$SCR - PRINTER SUPPORTS SCREENED PRINTING..... 2-17
2.3.11 PD$REV - PRINTER SUPPORTS REVERSED PRINTING.....2-17
2.3.12 PD$GRY - PRINTER SUPPORTS GRAY PRINTING..... 2-17
2.3.13 PD$GRF - PRINTER SUPPORTS GRAPHICS FUNCTIONS.....2-17
2.3.14 PD$LED - PRINTER HAS LEADING CONTROL.....2-18
2.3.15 PD$FNT - PRINTER HAS SELECTABLE FONTS..... 2-18
2.3.16 PD$WH1 - PRINTER SUPPORTS TYPE 1 PRINTWHEELS.....2-18
2.3.17 PD$WH2 - PRINTER SUPPORTS TYPE 2 PRINTWHEELS.....2-18
2.3.18 PD$TOP - PRINTER HAS TOP-OF-CHARACTER ORIENTATION.....2-19
2.3.19 PD$EDG - PRINTER CANNOT PRINT AT EDGE OF PAGE.....2-19
2.3.20 PD$IMG - PRINTER CAN PROCESS IMAGES.....2-19
2.3.21 PD$SFC - PRINTER HAS SELECTABLE FONT CARTRIDGES.....2-19
2.3.22 PD$KRN - PRINTER SUPPORTS TRACK KERNING.....2-20
2.3.23 PD$KEY - PRINTER SUPPORTS KEYCAP PRINTING..... 2-20
2.3.24 PD$W20 - PRINTER IS AlphaWRITE 2.0 COMPATIBLE.....2-20
2.3.25 PD$W21 - PRINTER IS AlphaWRITE 2.1 COMPATIBLE.....2-20
2.3.26 PD$ISO - PRINTER SUPPORTS ISO LATIN-1 CHARACTER SET..... 2-20
2.4 PRINTER DRIVER FUNCTIONS..... 2-21
2.4.1 P.INI - INITIALIZE PRINTER..... 2-21
2.4.2 P.CLS - CLOSE (SHUTDOWN) PRINTER.....2-21
2.4.3 P.CHR - OUTPUT CHARACTER.....2-21
2.4.4 P.CTL - PERFORM CONTROL FUNCTION.....2-22
2.4.5 P.SPL - Output Special Control String..... 2-23
2.4.6 P.CRLF - MOVE TO NEXT LINE..... 2-23
2.4.7 P.TOF - MOVE TO TOP OF FORM (START NEXT PAGE).....2-23
2.4.8 P.PSON - ENABLE PROPORTIONAL SPACING.....2-23
2.4.9 P.PSOF - DISABLE PROPORTIONAL SPACING..... 2-24
2.4.10 P.UNDR - TOGGLE UNDERSCORE.....2-24
2.4.11 P.BOLD - TOGGLE BOLD..... 2-24
2.4.12 P.STRK - TOGGLE STRIKEOUT.....2-24
2.4.13 P.BAR - TOGGLE OVERBAR.....2-24
2.4.14 P.SLPI - SET VERTICAL MOTION INDEX.....2-24
2.4.15 P.SHMI - SET HORIZONTAL MOTION INDEX..... 2-25
2.4.16 P.SCPI - SET CHARACTERS PER INCH (PITCH)..... 2-25
2.4.17 P.STM - SET TOP MARGIN..... 2-25
2.4.18 P.MTM - MOVE TO TOP MARGIN..... 2-25
2.4.19 P.SLPO - SET LEFT PAPER offset.....2-25
2.4.20 P.MLPO - MOVE TO LEFT PAPER OFFSET.....2-26
2.4.21 P.SLPP - SET LINES PER PAGE.....2-26
2.4.22 P.SLSP - SET LINE SPACING.....2-26
2.4.23 P.OVRP - OVERPRINT..... 2-26
2.4.24 P.LF - OUTPUT MULTIPLE LINE FEEDS..... 2-27
2.4.25 P.DBL - TOGGLE DOUBLE UNDERSCORE..... 2-27
2.4.26 P.FONT - SELECT FONT FAMILY.....2-27
2.4.27 P.ECHR - OUTPUT EXTENDED CHARACTERS..... 2-28
2.4.28 P.PSIZ- SET PAGE SIZE IN POINTS.....2-28
2.4.29 P.TMAR - SET TOP MARGIN IN POINTS.....2-28
2.4.30 P.LEFT - SET LEFT MARGIN IN POINTS.....2-28
2.4.31 P.LEAD - SET LEADING..... 2-29
2.4.32 P.FNAM - SET FONT BY NAME..... 2-29

2.4.33 P.FSIZ - SET FONT SIZE....2-29
2.4.34 P.ITAL - TOGGLE ITALICIZED PRINTING....2-29
2.4.35 P.SCRN - TOGGLE SCREENED PRINTING....2-30
2.4.36 P.REV - TOGGLE REVERSED PRINTING....2-30
2.4.37 P.FATR - SET FONT ATTRIBUTES DIRECTLY....2-30
2.4.38 P.MCHR - MOVE TO CHARACTER POSITION....2-30
2.4.39 P.MLIN - MOVE TO LINE POSITION IN D2....2-31
2.4.40 P.MOVE - DIRECT MOVE TO COORDINATE LOCATION ON PAGE..2-31
2.4.41 P.SAVE - SAVE PRINT STATE....2-31
2.4.42 P.REST - RESTORE PRINT STATE....2-31
2.4.43 P.FTBL - RETURN INDEX TO FONT TABLE....2-31
2.4.44 P.BYTE - DIRECTLY OUTPUT BYTE....2-31
2.4.45 P.SPOR - SET PAGE ORIENTATION....2-32
2.4.46 P.GRAY - SET GRAY SHADING....2-32
2.4.47 P.GRAF - PERFORMS GRAPHICS FUNCTION....2-32
2.4.48 P.FSUB - PERFORM FONT SUBSTITUTION....2-33
2.4.49 P.TRSC - TRANSLATE SPECIAL CHARACTER....2-33
2.4.50 P.TREC - TRANSLATE EXTENDED CHARACTER....2-33
2.4.51 P.OTSC - OUTPUT SPECIAL CHARACTER....2-33
2.4.52 P.OTEC - OUTPUT EXTENDED CHARACTER....2-34
2.4.53 P.SETL - SET LINE WEIGHT....2-34
2.4.54 P.POVR - OUTPUT PAGE OVERLAY....2-34
2.4.55 P.IMON - IMAGE PROCESSING ON....2-34
2.4.56 P.IMOF - IMAGE PROCESSING OFF....2-35
2.4.57 P.KEY - TOGGLE KEYCAP PRINTING....2-35
2.4.58 P.CART - SET CARTRIDGE/PRINTWHEEL/THIMBLE....2-35
2.4.59 P.KERN - SET TRACK KERNING....2-35
2.4.60 P.SPAG - SET PAGE PRINTING....2-36
2.4.61 P.OUTL - TOGGLE OUTLINE FONTS....2-36
2.4.62 P.CBAR - TOGGLE CHANGE BARS....2-36
2.4.63 P.PCH - RETURN PRINTER CHARACTERISTICS....2-36
2.4.64 P.DDPE - TOGGLE DEVICE-DEPENDENT PRINT EFFECT....2-37
2.5 FONT TABLE....2-37
2.5.1 TABLE STRUCTURE....2-37
2.5.2 FONT ENTRY STRUCTURE....2-37
2.5.2.1 FO.LOG - Logical Name Of Font....2-37
2.5.2.2 FO.CAR - Cartridges Supporting Fonts....2-38
2.5.2.3 FO.PHY - Physical Name Of Font....2-38
2.5.2.4 FO.FLG - Font Flags....2-38
2.5.2.5 FO.TBL - Name Of Font Width Table....2-38
2.5.2.6 FO.SIZ - Font Size In Points....2-38
2.5.2.7 FO.LED - Leading In Points....2-38
2.5.2.8 FO.CPI - Characters Per Inch....2-39
2.6 FONT WIDTH TABLES....2-39
2.6.1 AFM FORMAT FONT WIDTH TABLES....2-39
2.6.1.1 Flags Longword....2-39
2.6.1.2 Character Spacing Table....2-40
2.6.1.3 Pair Kerning Table....2-40
2.6.2 CENTIPOINTS-fORMAT FONT WIDTH TABLES....2-40

CHAPTER THREE - COMMAND LINE PROCESSING

WHAT IS COMMAND LINE PROCESSING?.....3-1
HOW COMMAND LINE PROCESSING WORKS.....3-1
COMMAND LINE FORMAT.....3-2
COMMAND LINE OPTIONS.....3-2
COMMAND LINE SYNTAX.....3-3
Commas.....3-3
Type-Ahead.....3-4
Using Caret (^) and Apostrophe (') as Text Characters.....3-4
Cancel Command Processing.....3-4
Error Handling.....3-4
COMMAND LINE PROCESSING SAMPLE.....3-5
USING SCRIPT FILES.....3-5
Chaining Script Files.....3-6
Pausing Script Files.....3-6
Using Colons.....3-7
File Example to Use with /F: Option.....3-7
AlphaBASIC PROGRAM.....3-8

CHAPTER FOUR - INSTRUCTIONS FOR TRANSLATING MESSAGE FILES

MESSAGE FILE NAMES.....4-1
OVERVIEW OF MESSAGE FILE CONTENTS.....4-2
HOW AlphaWRITE PROCESSES A MESSAGE FILE.....4-2
INITIAL ASSEMBLER DIRECTIVES.....4-3
DISPATCH AREA.....4-4
MESSAGE AREA.....4-5
The Message Dispatch Area.....4-5
The Message Table Area.....4-5
The CRT Statement.....4-7
The POS Statement.....4-7
Mode vs. Field Terminals.....4-7
The BYTE Statement.....4-8
The ASCII Statement.....4-9
The ASCIZ Statement.....4-9
Translating The Message Area.....4-10
Example One.....4-10
Example Two.....4-11
Example Three.....4-12
COMMAND AREA.....4-12
Command Dispatch Area.....4-13
Command Table Area.....4-13
Translating the Command Table Area.....4-13
FIELD AREA.....4-14
Field Dispatch Area.....4-14
Field Table Area.....4-14
Translating the Field Dispatch Area.....4-15
FINAL ASSEMBLER DIRECTIVES.....4-15
ASSEMBLING A MESSAGE FILE.....4-16
HELP FILES.....4-16

CHAPTER FIVE - EXTENDED CHARACTERS
DEFINING AN EXTENDED CHARACTER SET.....5-1
CREATING AN EXTENDED CHARACTER DEFINITION TABLE.....5-1
MAPPING EXTENDED CHARACTERS TO THE KEYBOARD.....5-2
An Extended Character Example..... 5-2
SCOPE OF EXTENDED CHARACTERS.....5-3

CHAPTER SIX - THE AWREAD XCALL

CHAPTER SEVEN - INTERNATIONAL LANGUAGE SUPPORT: NRCS TO ISO LATIN-1 TRANSFORMATION
DOCUMENT CONVERSION.....7-1
TRANSFORMATION TABLE FORMAT..... 7-2
SAMPLE TRANSFORMATION TABLE..... 7-2

CHAPTER EIGHT - CHARACTER COMPOSITION AND ISO LATIN-1 SUPPORT
ENTERING ISO CHARACTERS VIA THE TERMINAL.....8-1
ENTERING ISO CHARACTERS VIA AlphaWRITE SOFTWARE..... 8-1
HOW CHARACTERS ARE DISPLAYED.....8-2
HOW CHARACTERS ARE PRINTED.....8-2
PostScript Laser Printing..... 8-2
HP Laser Printing.....8-2
Substitution.....8-3
Printers Not Supported..... 8-3
LANGUAGE DEFINITION FILES..... 8-3
CHARACTER SET TRANSLATION TABLES.....8-3
HOW AlphaWRITE DETERMINES 7-BIT VS. 8-BIT TERMINALS.....8-3
SUMMARY 8-4

CHAPTER NINE - CONJUGATING VERB DICTIONARY..... 9-1
THE VERB DICTIONARY.....9-1
DICTIONARY LOOK-UP SEQUENCE..... 9-2
THE CONJUGATION TABLE..... 9-2

APPENDIX A - SAMPLE PRINTER DRIVERS.....A-1
PDVSYM.M68 AND PRINTER DRIVER SOURCE FILE LOCATION A-1
PRINTER DRIVER SOURCE ROAD MAP A-1

APPENDIX B - THE ASCII AND ISO LATIN-1 CHARACTER SETS CHART

DOCUMENT HISTORY

INDEX

READERS' COMMENTS

PRODUCTION NOTES

We maintain AlphaWRITE's documentation—including all of its on-line help files—using the current version of the AlphaWRITE software. This edition was generated using AlphaWRITE 2.1A.

We print AlphaWRITE's documentation using an Apple LaserWriter printer and the LASWRT printer driver.

We print on-line help files to a disk file using a custom printer driver capable of outputting terminal display codes. This custom printer driver is a modified version of AlphaWRITE's Miscellaneous printer driver (MISC.PDV).

The title and first edition pages were generated by a PostScript program file.

The text body is set in 11 point Helvetica with some examples appearing in 10 point Courier. Other special print emphasis features we used are described in Chapter 1 - "Introduction."

CHAPTER ONE
INTRODUCTION TO AlphaWRITE PROGRAMMER'S INFORMATION

This *AlphaWRITE Programmer's Information* manual contains material specifically for programmers who want to access AlphaWRITE's files and capabilities from their own software, or modify AlphaWRITE's operation by changing the screen displays or by creating new printer drivers.

HOW THIS BOOK IS ORGANIZED

This book is organized into nine chapters and two appendices:

- Chapter 2: Printer Driver Standards
- Chapter 3: Command Line Processing
- Chapter 4: Foreign Language Support
- Chapter 5: Extended Characters and Foreign Language Support
- Chapter 6: The AWREAD XCALL
- Chapter 7: International Language Support: NRCS to ISO Latin-1 Transformation
- Chapter 8: Character Composition and ISO Latin-1 Support
- Chapter 9: Conjugating Verb Dictionary
- Appendix A: PDVSYM.M68 and Sample Printer Drivers
- Appendix B: The ISO Latin-1 Character Set

CONVENTIONS USED IN THIS MANUAL

To make this document as easy as possible to understand, we use several conventions in layout and type style. This makes it easier to recognize whether we are talking about a command or key, when you're to type an entry, and so on.

CONVENTION	EXPLANATION
COMMAND	AlphaWRITE command names are shown in all capitals, bold type in a smaller point size than main text.
COMMAND	AMOS command names and some AlphaWRITE file names are shown in all capitals, non-bold type and in the same point size as main text.
AlphaWRITE Main Menu	AlphaWRITE screen names are printed like the example: no bold, initial capitals.
`What You Type`	Text you are expected to type is shown in a bold Courier typeface.
`Your Terminal Displays`	Text displayed on your terminal is shown in a non-bold Courier typeface.
KEY	Keys you need to press are shown in a keycap box.
CTRL-KEY	Indicates a control-key sequence.
ESC KEY	Indicates an escape-key sequence.
	The halt symbol denotes an important piece of information you should read carefully before continuing. Usually, text next to this symbol contains instructions for something you MUST or MUST NOT do. Please, read carefully.
	The hint symbol denotes a helpful bit of information. It is often used to point out a shortcut which may save you time or trouble.
	The remember symbol denotes information you need to keep in mind while you are following a set of instructions.

THE AlphaWRITE DOCUMENTATION LIBRARY

This *AlphaWRITE Programmer's Information* is just one document in the AlphaWRITE Documentation Library. The other documents are:

- The *AlphaWRITE Release Notes* - this document describes enhancements or changes in the current version of AlphaWRITE from the previous version.
- The *AlphaWRITE Installation Instructions*, DSO-00151-00 - provides complete instructions for installing the AlphaWRITE software on your computer.
- The *AlphaWRITE Reference Guide* - DSO-00078-00: AlphaWRITE's main reference document contains step-by-step operating instructions for end users. Its appendices contain technical information especially for System Operators.
- *AlphaWRITE Keyboard Commands Reference Booklet* - DSS-10316-00: This reference booklet lists the terminal keys you use to invoke AlphaWRITE features.

If you want to order an AlphaWRITE Documentation Library and receive the slip case and binder with the documentation, please order part number PDB-00004-03.

CHAPTER TWO
EXTENDED PRINTER DRIVER SPECIFICATION

A printer driver is a collection of subroutines that perform printer functions. By grouping all of the printer-specific control routines in a separate file, applications such as AlphaWRITE are able to support a wide variety of printers. New printers can be supported by writing a new printer driver, without any need to change the application software.

This chapter describes the printer driver standard, past and present, with emphasis on AlphaWRITE as the target application.

Due to the length and technical nature of this chapter, it contains numbered section headings, unlike other chapters in this document or other AlphaWRITE documentation.

2.1 PRINTER DRIVER FUNDAMENTALS

A printer driver is a binary file with the extension .PDV. Printer drivers must reside in system memory, user memory, or DSK0:[1,6] in order to be located by applications.

AlphaWRITE comes with the .M68 source files for all of the .PDVs that Alpha Micro supports. The .PDV file is created by assembling its .M68 source file with the Macro Assembler that accompanies the AMOS 1.4 or 2.x software.

Appendix A discusses supported printers and the printer drivers associated with them.

2.1.1 CREATING A NEW PRINTER DRIVER

To create a new printer driver, you should always start by copying an existing one. Choose the printer driver that most closely resembles the new printer in capabilities. The printer drivers included with AlphaWRITE are shown below.

PDV NAME	PRINTER
LASWRT	Apple LaserWriter and other PostScript printers
LAS*	HP LaserJet, LaserJet+, LaserJet II, IID, IIP, III, and IIID
PSCART	HP LaserJet models with PostScript cartridge installed

PDV NAME	PRINTER
AM301	AM-301 (NEC SpinWriter) thimble printer
AM302	AM-302 (Diablo 630) daisy-wheel printer
AM306?	AM-306 dot matrix printer
MISC	Miscellaneous (fine for most dot matrix printers)

2.1.2 PDVSYM.M68

Printer drivers include a file named PDVSYM.M68. This file is included with Alpha-WRITE and should reside in DSK0:[7,7] (MAC:). Printer drivers include this file during assembly using the COPY directive:

```
COPY      PDVSYM
```

See Appendix A for more information about `PDVSYM.M68`.

2.2 PRINTER DRIVER STRUCTURE

Printer drivers must adhere to the structure defined in this section in order to perform correctly.

2.2.1 ENTRY POINTS

An overview of the printer driver structure is shown below. Each symbol name represents an entry point into the printer driver.

NAME	DESCRIPTION	SIZE
P.HDR	Program Header	10. bytes
P.FLG	Flags	4 bytes
P.INI	Vector to PDINI routine	4 bytes
P.CLS	Vector to PDCLS routine	4 bytes
P.CHR	Vector to PDCHR routine	4 bytes
P.CTL	Vector to PDCTL routine	4 bytes
P.SPL	Vector to PDSPL routine	4 bytes
P.CRLF	Vector to PDMNLN routine	4 bytes
P.TOF	Vector to PDTOF routine	4 bytes
P.PSON	Vector to PDPSON routine	4 bytes
P.PSOF	Vector to PDPSOF routine	4 bytes
P.UNDR	Vector to PDUNDR routine	4 bytes
P.BOLD	Vector to PDBOLD routine	4 bytes
P.STRK	Vector to PDSTRK routine	4 bytes
P.BAR	Vector to PDBAR routine	4 bytes
P.SLPI	Vector to PDSLPI routine	4 bytes
P.SHMI	Vector to PDSHMI routine	4 bytes

NAME	DESCRIPTION	SIZE
P.SCPI	Vector to PDSCPI routine	4 bytes
P.STPM	Vector to PDSTPM routine	4 bytes
P.MTPM	Vector to PDMTPM routine	4 bytes
P.SLPO	Vector to PDSLPO routine	4 bytes
P.MLPO	Vector to PDMLPO routine	4 bytes
P.SLPP	Vector to PDSLPP routine	4 bytes
P.SLSP	Vector to PDSLSP routine	4 bytes
P.OVRP	Vector to PDOVRP routine	4 bytes
P.LF	Vector to PDLF routine	4 bytes
P.DBL	Vector to PDDBL routine	4 bytes
P.FONT	Vector to PDFONT routine	4 bytes
P.ECHR	Vector to PDECHR routine	4 bytes
P.SPARE	Reserved by Alpha Micro	20. bytes
P.USER	Reserved for users	32. bytes
*P.PSIZ	Vector to PDPSIZ routine	4 bytes
*P.TMAR	Vector to PDTMAR routine	4 bytes
*P.SPAR2	Reserved by Alpha Micro	12. bytes
*P.LEFT	Vector to PDLEFT routine	4 bytes
*P.LEAD	Vector to PDLEAD routine	4 bytes
*P.FNAM	Vector to PDFNAM routine	4 bytes
*P.FSIZ	Vector to PDFSIZ routine	4 bytes
*P.ITAL	Vector to PDITAL routine	4 bytes
*P.SCRN	Vector to PDSCRN routine	4 bytes
*P.REV	Vector to PDREV routine	4 bytes
*P.FATR	Vector to PDFATR routine	4 bytes
*P.MCHR	Vector to PDMCHR routine	4 bytes
*P.MLIN	Vector to PDMLIN routine	4 bytes
*P.MOVE	Vector to PDMOVE routine	4 bytes
*P.SAVE	Vector to PDSAVE routine	4 bytes
*P.REST	Vector to PDREST routine	4 bytes
*P.FTBL	Vector to PDFTBL routine	4 bytes
*P.BYTE	Vector to PDBYTE routine	4 bytes
*P.SPOR	Vector to PDSPOR routine	4 bytes
*P.GRAY	Vector to PDGRAY routine	4 bytes
*P.GRAF	Vector to PDGRAF routine	4 bytes
*P.FSUB	Vector to PDFSUB routine	4 bytes
*P.TRSC	Vector to PDTRSC routine	4 bytes
*P.TREC	Vector to PDTREC routine	4 bytes
*P.OTSC	Vector to PDOTSC routine	4 bytes
*P.OTEC	Vector to PDOTEC routine	4 bytes
*P.SETL	Vector to PDSETL routine	4 bytes
*P.POVR	Vector to PDPOVR routine	4 bytes
*P.IMON	Vector to PDIMON routine	4 bytes
*P.IMOF	Vector to PDIMOF routine	4 bytes
*P.KEY	Vector to PDKEY routine	4 bytes
*P.CART	Vector to PDCART routine	4 bytes
*P.KERN	Vector to PDKERN routine	4 bytes
*P.SPAG	Vector to PDSPAG routine	4 bytes

NAME	DESCRIPTION	SIZE
*P.OUTL	Vector to PDOUTL routine	4 bytes
*P.CBAR	Vector to PDCBAR routine	4 bytes
*P.PCH	Vector to PDPCH routine	4 bytes
*P.DDPE	Vector to PDDDPE routine	4 bytes

* Entry points marked with an asterisk apply to **extended printer drivers** only. See section 2.2.4 for a definition of an extended printer driver.

This structure is followed by the various routines themselves. The size of each routine varies.

2.2.2 PROGRAM HEADER

The program header area, `P.HDR`, is the standard AMOS program header used in many other Alpha Micro files. The program header contains a number of flags, as well as the version number. In the printer driver source file, the `PHDR` macro is used to generate the 10. byte header:

```
PHDR    -1,0,PH$REE!PH$REU      ; program header
```

2.2.3 CAPABILITY FLAGS

The flags longword, `P.FLG`, contains a number of bit flags that define the printer's capabilities and attributes to the calling application. It is extremely important that these flags accurately inform AlphaWRITE about what the printer driver can do, otherwise AlphaWRITE will not be able to make intelligent decisions during the printing process.

The printer driver flags use symbol names of the form `PD$xxx`, and are described below.

FLAG	MEANING
PD$VAR	Has variable spacing
PD$PRO	Has proportional spacing
PD$FED	Supports cut sheet feeder(s)
*PD$EXT	Is an extended printer driver
*PD$PTS	Uses points as a natural unit of measurement
*PD$SAV	Can save/restore print state
PD$BLD	Supports the bold print feature
PD$UND	Supports the underscore print feature
*PD$ITL	Supports the italic print feature
*PD$SCR	Supports the screened print feature
*PD$REV	Supports the reversed print feature
*PD$GRY	Supports the gray print feature
*PD$GRF	Supports graphics functions
*PD$LED	Has leading (vertical spacing) control

FLAG	MEANING
*PD$FNT	Has selectable fonts
*PD$WH1	Uses type 1 (AM301) printwheels
*PD$WH2	Uses type 2 (AM302) printwheels
*PD$TOP	Orients at top-of-printhead instead of baseline
*PD$EDG	Can't print at edge of paper
*PD$IMG	Can print image bitmaps
*PD$SFC	Has selectable font cartridges
*PD$KRN	Supports track kerning
*PD$KEY	Supports the keycap print feature
*PD$W20	Is an AlphaWRITE 2.0 compatible printer driver
*PD$W21	Is an AlphaWRITE 2.1 compatible printer driver
*PD$ISO	Supports the ISO Latin-1 character set
*PD$DSP	Supports double-sided printing
*PD$ROT	Supports text rotation
*PD$OUT	Supports outline fonts
*PD$CHG	Supports change bars
*PD$PCH	Can return detailed characteristics

*Flags marked with an asterisk apply to **extended printer drivers** only. See the section "Extended Printer Drivers" for a definition of an extended printer driver.

2.2.4 EXTENDED PRINTER DRIVERS

Prior to the release of AlphaWRITE 2.0, a printer driver standard existed, now known as the *traditional printer driver standard*. Concurrent with the release of AlphaWRITE 2.0, this standard was enhanced and revised, and is now known as the *extended printer driver standard*.

The extended printer driver standard provides a far better standard interface between an application program and the printer driver, especially since the introduction of printers with a wealth of built-in features and functions. The traditional printer driver dealt with traditional units of measurement, such as lines and character positions. But with the advent of proportional fonts and variable line spacing, these traditional units are no longer appropriate. Indeed, the interactions of printer settings and driver commands are too complex to achieve usable and reproducible results using traditional units of measurement; continuing to use them will result in frustration for both the printer driver programmer and the end-user.

The extended printer driver standard specifies units in terms of *points* (72 points = 1 inch). This is an unambiguous unit in contrast to line spacing (which depends on the current vertical motion index setting) or character positioning (which depends on the current horizontal motion index setting or the current pitch).

An extended printer driver is identified by setting the `PD$EXT` flag in the `P.FLG` flags longword in the printer driver header.

2.2.4.1 Compatibility

Older software, written before the extended printer driver standard was published, will only use the line/character calls. These programs will not work if you only implement the points-oriented functions and do not implement the line/character functions in your printer driver. The difficulty in implementing the line/character functions is that the interactions between various settings are not well defined. For example, the `P.STM` function sets the number of lines between the top of form and the first title line on the page. Its interaction with the `P.SLSP` function (which sets the line spacing) and the `P.SLPI` function (which sets the vertical motion index, or the "lines per inch") has never been defined. Indeed, different printer drivers will give different results when the `P.MTM` (move to top margin) function is called. The position of the print head could change from printer driver to printer driver.

2.2.4.2 Implementation Considerations

You do not need to have a printer that uses points as its natural unit for either horizontal or vertical movement in order to use or develop an extended printer driver. You can signal that your printer driver can handle points-oriented functions by setting the `PD$PTS` flag (and the `PD$EXT` flag) in the printer driver header, and then implement as many of the points-oriented functions as possible. Inside each function, you convert the points value supplied to the function into the natural unit of measurement that the printer requires, and then use that value.

2.2.4.3 Recommendation

Your applications program should interrogate the printer driver header and check for the `PD$EXT` and `PD$PTS` flags. If it finds both these flags set, the program should use the points-oriented functions and pass parameters in points. It should not use the line/character-oriented calls and pass values in terms of lines or character positions.

Our strong recommendation is for you to develop applications software that use points-oriented calls, and to write printer drivers that implement the points-oriented functions and either make a "best attempt" at implementing the line/character functions or cause an error message to be printed if these functions are not implemented but get called by an application.

If the printer driver is an extended driver, it is expected to deal with all of the functions in section 2.2.1 that are marked with an asterisk. Likewise, any of the applicable flags in section 2.2.3 marked with an asterisk may be specified.

If the printer driver is not extended, it may not make use of the extended functions or flags under any circumstances.

2.2.5 VERSION INTERACTION

Printer drivers are complex entities and are subject to change. In the past, implementation of printer drivers depended on an intimate internal knowledge of AlphaWRITE. With AlphaWRITE versions 2.0 and later, a logical separation between application and driver has been maintained.

Using older printer drivers with AlphaWRITE 2.0 and later may provide unexpected results; to avoid these problems, two printer driver flags are used to identify version compatibility: `PD$W20` and `PD$W21`. Printer drivers that predate AlphaWRITE 2.0 should **not** set either of these flags. AlphaWRITE 2.0-compatible printer drivers should set the `PD$W20` flag. AlphaWRITE 2.1-and-later-compatible printer drivers should set the `PD$W21` flag. It is invalid for a printer driver to set **both** flags.

When AlphaWRITE 2.0 is invoked, it will only accept printer definitions when the referenced printer driver has the `PD$W20` flag set. AlphaWRITE 2.0 will not accept AlphaWRITE 2.1 and later printer drivers.

When AlphaWRITE 2.1 or later is invoked, it will accept printer definitions when the referenced printer driver has the `PD$W21` flag set. AlphaWRITE versions 2.1 and later support both 2.0 and 2.1 level printer drivers.

The actual version number of a printer driver, defined in its `PHDR` macro, does not come into play. AlphaWRITE consults only the `PD$W20` and `PD$W21` flags when deciding whether or not to accept a particular printer definition.

2.2.6 PRINTER DRIVERS AND YOUR APPLICATION

Applications and printer drivers interact according to specific principles. The following sections provide fundamental information for using a printer driver with your application.

The topics discussed are:

- Division of responsibility between the applications program and the printer driver.
- The printer driver's communication area.
- The printer driver's impure area.
- Using a printer driver in an application.
- Examples of using printer drivers.

2.2.6.1 Division Of Responsibility

It is the application's responsibility to issue all the commands to ensure characters are printed in the right place, in the right style, font, etc. The application must keep track of, and act on, issues such as moving the printhead to the left print margin, keeping track of the length of the printed line, and ending the line. Moving to the start of the next line, and keeping track of the vertical print position on the page, and the top and bottom margins are also an applications responsibility.

The printer driver handles the device-dependent parts only. There is a similarity in concept between printer drivers and terminal drivers. Both provide a device-independent interface to your applications program.

But in each case, the applications program has the responsibility of checking for implemented features and using the driver functions intelligently.

2.2.6.2 The Driver's Communication Area

The application passes parameters to the printer driver by passing values in registers and sometimes by passing additional information in a memory block. This memory block is called the Driver Communications Area. The format of the communications area is fixed and must not be changed; it is defined in `PDVSYM.M68`.

When a printer driver function is called, register A3 must point to the first byte of the communications area.

2.2.6.3 The Driver's Impure Area

If the driver needs an impure area, one can be specified inside the driver's communications area as discussed below.

Developers can set up printer driver impure areas by defining variables using the `.OFDEF` command in the printer driver source file. The `.OFINI` command used to define the start of the area must specify `RESVUS` (defined in `PDVSYM.M68` as a location in the communications area specially reserved for this task) as the address of the start of the impure area. The `.OFSIZ` command used to terminate the impure area should specify the variable `PDVMEM` to hold the size of the area. The size of the impure area must not exceed 250. bytes. An assembler-check should be inserted in the source code to ensure this limit is not exceeded. This is done with a conditional assembly of an `ASMERP` instruction with the help of a symbol (`PDVEXT`) defined in `PDVSYM.M68`.

As an example, to define an impure area consisting of a floating point variable (`FPT`) and a longword (`LWD`), the following code would be used:

```
        COPY       PDVSYM
        OBJNAM     0,0,[PDV]

        .OFINI     RESVUS
        .OFDEF     FPT, 6.
        .OFDEF     LWD, 4.
        .OFSIZ     PDVMEM
IF LT,             PDVEXT-PDVMEM
        ASMERP     "?PDV impure area is too large!"
ENDC
        ....
```

2.2.6.4 Using A Printer Driver In An Application

The applications program has to perform a number of tasks in order to successfully use a printer driver. The exact details of what is required varies from program to program, but the following outline should provide some guidance:

- **Get the driver name.** The name of the required printer driver must be obtained from the user or from the application.

- **Load the `.PDV` into memory.** The printer driver must be loaded into memory using a documented, application-defined search path. AlphaWRITE searches user memory, then system memory, then DSK0:[1,6].

- **Check the .PDV's features.** The `P.FLG` flag longword in the printer driver header should be interrogated to see what features are available. If the right set of features is not available, the user should be notified and the appropriate action taken.

- **Allocate buffers for font width tables.** Depending on the header information, some applications may need to allocate extra buffer space to hold other file information, such as font width tables, that are not part of the printer driver.

- **Allocate buffers for the application's use.** Some applications may need to allocate one or more line buffers to hold text as it is being constructed. The application will pass the contents of the buffer, character by character, to the printer driver. Typically, simple requirements such as monospaced fonts with a ragged right margin will not require buffers. However, applications using kerning, proportional fonts, right justification, or multiple fonts will probably need the buffers to build up lines and interrogate font width tables before passing the formatted lines to the printer driver a character at a time.

- **Allocate the comms area.** The communications area must be allocated and nulled out.

- **Initialize the comms area.** The application should set up the spooling flags in `SPLPOS(A3)` and `SPLNEG(A3)` in the printer communications area. The printer driver is allowed to consult these flags and alter them if it thinks it appropriate. This is usually done in the `P.CLS` function.

- **Set up the output file.** The output file DDB must be set up, and the buffer address linked to it. The output DDB is at `PTDDB(A3)` and its buffer is at `PTBUF(A3)` in the communications area.

- **Open the output file.** The output file must be opened for output, possibly deleting any existing file of the same name.

At this point, the driver is ready to process its first call.

2.2.6.5 Register Calling Conventions

Whenever the applications program calls a function, register A3 must point to the first byte of the communications area, not the first byte of the impure area. Registers D6, D7 and A6 can be used freely by the printer driver. Other registers should be saved and restored in the driver's function. Although it is the printer driver's responsibility to save any other registers it uses, it is good defensive applications programming to save all non-volatile registers before calling the driver. As an example, to call the printer driver's initialization routine, the code in the applications program might look like this (assuming A4 indexes the printer driver module):

```
SAVE        D0-D5, A0-A5
CALL        P.INI(A4)
REST        D0-D5, A0-A5
```

This will cope with ill-mannered printer drivers that do not save and restore all the registers they use.

2.2.6.6 Driver Initialization

Every printer driver must be initialized before any other function is used.

The first call to the printer driver must be to the `P.INI` function. This function should initialize the printer to a known state by outputting appropriate commands, as well as initializing its impure area. Remember the impure area has already been allocated and the output file opened before the `P.INI` function was called.

2.2.6.7 Outputting Characters

To print lines of text, the application calls the `P.CHR` function on a character by character basis. One character is passed from the application, through the printer driver, and out to the output file on each call to `P.CHR`.

The `P.CHR` function can deal correctly with any printable character in the ASCII character set. It can also handle printable characters from the ISO Latin-1 character set if the `PD$ISO` bit is set in the `P.FLG` flag longword in the driver. To output a special character (defined as a character accessed through AlphaWRITE's SPEC CHAR or by AlphaWRITE's compose character feature), the applications program must call the `P.OTSC` function instead of the `P.CHR` function. To output extended characters, the `P.OTEC` function should be called.

2.2.6.8 Outputting New Lines

The printer driver should move from one line to another by calling the `P.MOVE` command, and not by passing `CR` (carriage return) or `LF` (line feed) bytes to the `P.CHR` routine.

2.2.6.9 Outputting New Pages

The printer driver should move to the next page by calling the `P.TOF` routine, not by passing a `FF` (form feed) byte to the `P.CHR` routine. Subseqeuent movement to the first print line position should be done by calling the `P.MTM` function, not by using the `P.MOVE` command. However, the driver may implement the `P.MTM` function internally by using the `P.MOVE` function.

2.2.6.10 Finishing Up

The applications program must carry out the following functions at the end of the program:

- **Call `P.CLS`.** The last call to the printer driver should be to the `P.CLS` function. The `P.CLS` function can alter the spooling flags. See the section on the `P.CLS` function for details.
- **Close the output file**. The output file must be closed.
- **Spool the output file.** Optionally, the output file should be spooled.
- **Deal with comms area.** The communications area should be either nulled out or deallocated.
- **Deal with other allocated areas.** Any other print-related memory allocations (such as font width tables) can be deallocated.

2.2.6.11 Examples Of Using .PDV Calls

To show how an application would make use of a printer driver, a series of examples is given below. The examples assume you are using a points-oriented driver. Each example builds on the previous one to show how additional features can be implemented. The examples just indicate the outline of the procedure you need to follow; there is some other lower-level work that also needs to go on.

A Basic Implementation

Let's assume the `P.INI` call will set the printer into a monospaced font—for example, 10 point Courier. The basic implementation just outputs text with a ragged right margin.

- **Set things up.** The application loads the required printer driver and sets up the printer communications area, including the default spooling flags `SPLPOS(A3)` and `SPLNEG(A3)`, and opening the output file for output.
- **Initialize the driver**. The application calls the driver's `P.INI` function.
- **Set initial printer settings.** The applications program sets up the initial printer settings. For example, it might call these functions:
  - `P.TMAR` to set the top margin
  - `P.LEFT` to set the left margin
  - `P.PSIZ` to set the page length
  - `P.SLSP` to set the line spacing
- **Set up application variables.** The application program initializes its own variables that keep track of such things as margins, paper size, font and font size, print attributes, and print head position.

- **Move to top margin.** The first line of text can now be output. The application moves the print head to the top margin position with the `P.MTM` call, and updates its own internal variables.

- **Output characters.** The application can now output characters, one at a time, by calling the `P.CHR` call. Extended or special characters can be output by the `P.OTEC` and the `P.OTSC` calls respectively. The applications program updates its print co-ordinates variables.

- **Outputting highlighted text.** To output, say, bold text, the application can call a printer driver function (the `P.BOLD` function in this case) just before the first character to be highlighted is output. Characters are then outputted as usual. The applications program would call the same function again at the end of the highlighted text.

  The application should check the appropriate flags in the `P.FLG` longword in the driver's header before using the wanted function to ensure that the function is supported by the printer and the printer driver.

- **Indenting text**. Indentation can be achieved by the application calling the `P.MOVE` command. Again, the application program must update its internal variables.

- **Move to next line.** At the end of the line, the application issues a `P.MOVE` command to move the print head down to the next line and to the left print margin. Again, the application program must track this movement in its own variables.

  For the greatest portability between printers, all printhead movement should be down the paper.

- **Move to next page.** At the end of the page, the application calls the `P.TOF` routine if there is more text to be output. The applications program must track this operation in its own internal variables.

- **Call P.CLS.** After outputting all the text, the applications program calls the `P.CLS` routine.

- **Deal with output file.** The application closes the output file. Optionally, it could also spool the output file.

Refinement #1 - Simple Right Justification

To achieve right justification on a simple printer using a monospaced font, the application needs to know the desired length of the line (in points) and the width of a character (in points). The application needs to have a number of variables to keep track of present page and printer settings, and the current print position. Then, it would:

- **Build up the line in the buffer....** Add words and intervening spaces into a buffer, accumulating the line's length so far, until adding a word overflows the buffer.

- **....But not too much.** The last word and trailing space would be backed out.

- **Calculate additional spaces required.** The number of additional spaces would be calculated, and that number of spaces inserted into the buffer, one after each space already present until the correct number of additional spaces is inserted.

- **Prepare to output line.** The application would consult its internal print co-ordinates variables to see if the line about to be printed would fit on the current page. If not, it would issue commands to end the current page and start the next page (probably using calls to the `P.TOF` and `P.MTM` functions). In either case, the application would update its internal variables that track the current print co-ordinates.

- **Move printhead to next line.** The application would then issue a `P.MOVE` command to move the print head to the start of the next line including the left print margin. The printer may be able to make such a move directly if direct cursor addressing is possible. Otherwise, the driver has to always keep track of its current horizontal and vertical co-ordinates, calculate the required commands to move the print head to the desired position, and issue those commands. The printer updates its own internal co-ordinate variables as necessary.

- **Output the line, character by character.** The applications program would call the `P.CHR` function for each ASCII or ISO Latin-1 character in the buffer. It would call the `P.OTEC` function to output extended characters and would call the `P.OTSC` function to output special characters.

Refinement #2 - Adding Proportional Spacing Support

If a printer can move the printhead in smaller intervals than one character's width (i.e. if the `PD$PRO` bit in the `P.FLG` longword is set), the interactions may go like this:

- **Fill buffer.** The application fills the buffer to the desired length,

- **Calculate additional space size required.** It evaluates the amount of space remaining on the line (in points) and counts the number of space characters in the text on the line. It divides the amount of space left by the number of spaces on the line, and the result is the amount of extra head movement required whenever a space is printed.

- **Put answer in comms area.** The application places the Alpha Micro 6-byte floating-point result in the communications area at `WRDSPC(A3)`.

- **Check and move to next line**. The application would check if the line would fit on the current page, call the appropriate driver functions, and update its own internal variables.

- **Issue `P.PSON` call.** Before sending the line to the driver character by character, the application issues a call to the `P.PSON` function. This should set a flag in the driver that will modify the action of the `P.CHR` call.

- **Output characters.** The application issues a `P.CHR` (or `P.OTEC` or `P.OTSC`) call for every character in the buffer. The `P.CHR` function outputs each character,

checking for a space character. When it finds one, it outputs the space, and then advances the print head `WRDSPC(A3)` points. This may involve converting the floating-point points value to another unit and representation in order to issue the correct command.

- **Issue `P.PSOF` call.** At the end of the buffer, and before issuing the `P.MOVE` command, the application issues a `P.PSOF` call.

Refinement #3 - Adding Support for Proportional Fonts

When you add the need for proportional fonts, you add another level of complexity. The application must have access to the font width table for the font being used.

- **Check capabilities of driver.** The application must find out if the printer and driver can support the desired font. The application issues a `P.FSUB` command specifying the font it wants to use, and checks the returned font family information. It must use the result as the selected font. The application is free to carry out font substitution (by trying again with another `P.FSUB` call) if it chooses, but the end result must be that it uses a font that the printer driver supports.

- **Scan font table and choose entry**. The application can then scan the printer driver's font table by issuing an `P.FTBL` call to locate the start of the table. The application can then look at each font entry in turn. It can check if a font entry refers to a proportionally-spaced font by checking the `FO$PRO` flag in the `FO.FLG` longword of the table entry. It can choose the correct font size and attributes, and obtain the name of the corresponding font width table from the font table. The format of the font table is shown in section 2.5.

- **Load font width table**. The application can then load the selected font width table to use for character width calculations.

- **Tell driver about decisions.** The application now issues commands to tell the printer driver what it is about to use. Typically, it would call the `P.FONT`, `P.FSIZ` and maybe `P.FATR` functions. The printer driver would send commands to be issued to the output file to select the right font and typesize, and then update its internal variables.

- **Build up line.** Now the application calculates the length of the line built up so far in the print buffer by interrogating the font width table for the width of each character in the buffer.

  The font width table contains entries for ASCII or ISO Latin-1 characters only. To get widths of printer dependent characters, such as extended or special characters, the applications program must call the `P.TREC` or the `P.TRSC` driver routines respectively. These functions return a character to the applications program, which must then interrogate the font width table with that supplied character.

- **Carry on as before.** When the desired length is reached, the rest of the calculations and steps shown earlier can be done.

Note that there may be a different font width table for each type of print emphasis—upright ("standard"), bold, italic and bold-italic. Your application may have to load other tables if these attributes are chosen, and consult the correct table for the attribute(s) currently invoked. This consultation and the handling of the loading is the responsibility of the applications program.

Refinement #4 - Adding Track Kerning

You can add track kerning support to the application if the `PD$KRN` bit is set in the `P.FLG` longword in the driver's header.

- The application issues a `P.KERN` call specifying the severity of kerning required before outputting the buffer.
- The driver is responsible for implementing the degree of "tightening-up" that is required from the supplied parameter. Note that there is currently no way for the driver to communicate the amount back to the applications program. The applications program needs to know that value in order to calculate the line lengths correctly. Until such a method is defined, applications and drivers should implement the following:
    - Light kerning decreases space by 5% from the "no kerning" values.
    - Medium kerning decreases space by 10% from the "no kerning" values.
    - Heavy kerning decreases space by 15% from the "no kerning" values.
- Because the printer driver does not have access to the font width table, the application must put the kerning amount for each character in `PDVARG(A3)` in the communications area before calling the driver's `P.CHR` function to output the character. The amount is an Alpha Micro 6-byte floating-point format value of the number of points of kerning.
- The driver's `P.CHR` routine is enhanced to move the print cursor by the amount specified in `PDVARG(A3)` after printing each character.

2.3 CAPABILITY FLAGS

This section explains the capability flags in detail.

2.3.1 PD$VAR - VARIABLE SPACING

`PD$VAR` was used to indicate support for variable spacing. It indicated the printer's ability to variably change the spacing between words. This was used with drivers for daisy-wheel and thimble printers which had the ability to control inter-word spacing in units of 1/120" and 1/48".

This flag is now obsolete, and has been replaced by the `PD$PRO` flag. It should not be used.

2.3.2 PD$PRO - PROPORTIONAL MOVEMENT

PD$PRO indicates the printer's ability to vary the amount of print head movement when printing a space character. Daisy-wheel, thimble, and laser printers typically set this flag. Dot matrix printers generally do not unless they support microspacing. See section 2.4.8 on the P.PSON print function for an explanation of the required function.

2.3.3 PD$FED - CUT SHEET FEEDER

PD$FED indicates support for cut sheet feeders. If AlphaWRITE sees this flag, it will pass bin and feeder commands to the driver. If this flag is not set, bin and feeder commands will be ignored, even if specified in the document.

2.3.4 PD$EXT - EXTENDED PRINTER DRIVER

PD$EXT indicates that the printer driver adheres to the new, extended format. All of the extended driver vectors must be present, even if they call functions that do nothing. Any of the new capability flags may also be specified in the flags longword as long as the corresponding function is fully implemented.

2.3.5 PD$PTS - PRINTER DRIVER IS POINTS-ORIENTED

The PD$PTS flag, if set, states that the printer driver is inclined to deal in points (72 points = 1 inch) instead of lines and characters. If this flag is set, AlphaWRITE will use the new points-oriented calls instead of the traditional line and character oriented calls. The following table shows the calls that are used.

FUNCTION	PD$PTS SET	PD$PTS NOT SET
Set top margin	P.TMAR	P.STM
Set left margin	P.LEFT	P.SLPO
Set page size	P.PSIZ	P.SLPP
Set leading	P.LEAD	P.SLPI

PD$PTS is an extended printer driver flag.

2.3.6 PD$SAV - PRINTER CAN SAVE/RESTORE PRINT STATE

The PD$SAV flag is set if the P.SAVE and P.REST functions, which save and restore the print state, are implemented. If this flag is set, AlphaWRITE assumes it can save its current context, go do something else on the page, and restore original conditions.

This is an extended printer driver flag.

2.3.7 PD$BLD - PRINTER SUPPORTS BOLD

This flag, if set, indicates that the printer driver implements the `P.BOLD` function, which toggles bold printing.

2.3.8 PD$UND - PRINTER SUPPORTS UNDERSCORE

This flag, if set, indicates that the printer driver implements the `P.UNDR` function, which toggles underscored printing.

2.3.9 PD$ITL - PRINTER SUPPORTS ITALICS

This flag, if set, indicates that the printer driver implements the `P.ITAL` function, which toggles italicized printing.

This is an extended printer driver flag.

2.3.10 PD$SCR - PRINTER SUPPORTS SCREENED PRINTING

This flag, if set, indicates that the printer driver supports screened printing. Screened printing is black text on a gray background. The printer driver must implement the `P.SCRN` call.

This is an extended printer driver flag.

2.3.11 PD$REV - PRINTER SUPPORTS REVERSED PRINTING

This flag, if set, indicates that the printer driver supports reversed printing. Reversed printing is white text on a black background. The printer driver must implement the `P.REV` call.

This is an extended printer driver flag.

2.3.12 PD$GRY - PRINTER SUPPORTS GRAY PRINTING

This flag, if set, indicates that the printer driver supports gray printing. The `P.GRAY` function, which sets the gray level, must be implemented.

2.3.13 PD$GRF - PRINTER SUPPORTS GRAPHICS FUNCTIONS

`PD$GRF` indicates that the printer driver can perform some or all of the eight graphics functions currently supported by AlphaWRITE. These functions are:

- Start line
- End line
- Start box
- End box

- Start rounded box
- End rounded box
- Start gray box
- End gray box

The `P.GRAF` function must be implemented if this flag is set. This is an extended printer driver flag.

2.3.14 PD$LED - PRINTER HAS LEADING CONTROL

Leading (pronounced *LEDding*) is the control of vertical spacing from the baseline of one line to the next. Most laser printers have the ability to position the print cursor on the page with point resolution or better. If the `P.LEAD` function is implemented in the driver, this flag should be set.

This flag should only be specified if the `PD$PTS` flag is set as well. This is an extended printer driver flag.

2.3.15 PD$FNT - PRINTER HAS SELECTABLE FONTS

The `PD$FNT` flag indicates that the printer offers more than one font to choose from. The `P.FONT` (set font) and `P.FTBL` (return index to font table) calls are implemented, and that font tables for each font are available.

This is an extended printer driver flag.

2.3.16 PD$WH1 - PRINTER SUPPORTS TYPE 1 PRINTWHEELS

Prior to the extension of the printer driver standard, printwheel support was built-in to AlphaWRITE. Tables to support printwheels were internal to AlphaWRITE, not the driver, and decisions about what character to output were made on the basis of driver name.

The extended printer driver standard removes this interdependency and allows any printer driver to make use of printwheels. The `PD$WH1` flag, if specified, indicates that the printer accepts thimbles for the AM-302 (Diablo 630) printer.

If it sees this flag, AlphaWRITE will display the appropriate thimble list for this printer when the user issues the F)ont command from the `QUEUE` screen.

`PD$WH1` is an extended printer driver flag.

2.3.17 PD$WH2 - PRINTER SUPPORTS TYPE 2 PRINTWHEELS

The `PD$WH2` flag, like the `PD$WH1` flag described above, was added to define a particular type of printwheel. This flag indicates that the printer accepts daisywheels for the AM-301 (NEC Spinwriter) printer.

If it sees this flag, AlphaWRITE will display the appropriate daisywheel list for this printer when the user issues the F)ont command from the `QUEUE` screen.

This is an extended printer driver flag.

2.3.18 PD$TOP - PRINTER HAS TOP-OF-CHARACTER ORIENTATION

AlphaWRITE and the extended printer drivers normally assume a base line orientation. In other words, the "print cursor" is positioned where the baseline of the character will appear.

Older printers, particularly dot matrix printers, use a top-of-printhead orientation. When writing a printer driver that orients itself to the top of the character bounding box, instead of the baseline, set the `PD$TOP` flag.

If `PD$TOP` is set, AlphaWRITE can adjust its character positioning so that requested margins can be honored as accurately as possible.

This is an extended printer driver flag.

2.3.19 PD$EDG - PRINTER CANNOT PRINT AT EDGE OF PAGE

Most laser printers cannot print at the very edge of the paper—there is usually a quarter-inch region at the outer perimeter that can't be accessed. Your applications program can adjust the commands it sends the printer driver if this flag is set. It can also be used as a hint to the applications program not to print very close to any edge of the paper: characters might otherwise be lost or extra pages output.

This is an extended printer driver flag.

2.3.20 PD$IMG - PRINTER CAN PROCESS IMAGES

This flag, if set, indicates that the `P.IMON` (image on) and `P.IMOF` (image off) functions are implemented.

If this flag is not set, AlphaWRITE will not pass image blocks from documents through to the printer.

This is an extended printer driver flag.

2.3.21 PD$SFC - PRINTER HAS SELECTABLE FONT CARTRIDGES

This flag indicates that cartridges are available for the printer, and that the `P.CART` function is implemented. The printer driver's font table should indicate which cartridge(s) contain which fonts.

With the printer drivers supplied with AlphaWRITE, only LASJET, LASII, LASIID, and LASIII specify this flag. AlphaWRITE `QUEUE` command's screen for F)ont will show the HP cartridge definitions when this flag is set.

This is an extended printer driver flag.

2.3.22 PD$KRN - PRINTER SUPPORTS TRACK KERNING

Track kerning is the application of a formula to a typeset line, yielding a tightening factor between characters. The `PD$KRN` flag indicates that the `P.KERN` function is implemented.

Printer drivers released with AlphaWRITE 2.1 and later implement track kerning; earlier printer drivers did not implement track kerning.

This is an extended printer driver flag.

2.3.23 PD$KEY - PRINTER SUPPORTS KEYCAP PRINTING

This flag indicates that the printer driver can print the keycap special effect, and that the `P.KEY` function is implemented.

This is an extended printer driver flag.

2.3.24 PD$W20 - PRINTER IS AlphaWRITE 2.0 COMPATIBLE

This flag indicates AlphaWRITE 2.x compatibility with specific support for AlphaWRITE 2.0 special characters.

2.3.25 PD$W21 - PRINTER IS AlphaWRITE 2.1 COMPATIBLE

This flag indicates AlphaWRITE 2.x compatibility with specific support for AlphaWRITE 2.1 special characters.

2.3.26 PD$ISO - PRINTER SUPPORTS ISO LATIN-1 CHARACTER SET

This flag informs AlphaWRITE to pass not only printable ASCII characters to the driver but also ISO Latin-1 characters in the range 160-255 decimal. If this flag is omitted and 8-bit ISO Latin-1 text characters are encountered in the document during printing, AlphaWRITE 2.1A will translate the ISO Latin-1 character into an ASCII character using a user-definable table set up in WRT405.*lng*. If this flag is set, 8-bit text characters are passed to the printer driver.

2.4 PRINTER DRIVER FUNCTIONS

This section describes the printer driver functions in detail.

2.4.1 P.INI - INITIALIZE PRINTER

Applications call `P.INI` exactly once before beginning a print job. `P.INI` is always the first printer function called.

The `P.INI` function is responsible for initializing the printer to a known state. This should include all of the following, where applicable:

- Setting orientation to portrait.
- Setting current typeface to Courier.
- Setting current type size to 10 point with 12 point leading.
- Turning off bold, underscore, italics.
- Cancelling any print effects that may have been left on.

The printer driver should also initialize any internal variables in this routine.

2.4.2 P.CLS - CLOSE (SHUTDOWN) PRINTER

Applications must call `P.CLS` when they are finished outputting. `P.CLS` is always the last printer function called, and it must only be called once by the application.

The `P.CLS` function, like `P.INI`, should leave the printer in a known, initialized state. Remember that the next application to print on the printer may or may not be using this printer driver.

`P.CLS` can override AlphaWRITE's spooling defaults. To do this, `P.CLS` must place flag values into the longwords `SPLPOS(A3)` and `SPLNEG(A3)`. To prevent AlphaWRITE from spooling the output file completely, place a `-1` in `SPLPOS(A3)`. This control is sometimes needed when cut-sheet feeders are used, or with speciality drivers not using the AMOS spooler. However, this function must not close the output file or deallocate any memory. That is the responsibility of the application program.

2.4.3 P.CHR - OUTPUT CHARACTER

Inputs: D1 - character to output

`P.CHR` is the character output routine, used to output a character to the output file. The character to be output is contained in the low order byte of register D1 and the DDB is located at `PTDDB(A3)` in the communications area.

If the printer driver implements proportional spacing or variable word spacing, the field `WRDSPC(A3)` contains the number of points required to advance after each space in order to right-justify the current line. This is an Alpha Micro 6-byte floating point format field.

If the printer driver implements kerning, the field `PDVARG(A3)` contains the number of points to advance or retreat the print head after each character is output. This is an Alpha Micro 6-byte floating point format field.

The simplest implementation of `P.CHR` is simply to output the character to the output file:

```
PDCHR:  FILOTB  PTDDB(A3)
        RTN
```

More sophisticated printer drivers will perform more complex processing, as many print features may or may not be simultaneously engaged while printing a character.

2.4.4 P.CTL - PERFORM CONTROL FUNCTION

Inputs: D1 - control function code

The `P.CTL` routine performs a special printer function based on a function code passed in register D1. The table below shows these functions and their associated function codes.

CODE	MEANING
PC$CHOME	return to carriage home
PC$MOVAHT	move to absolute horizontal tab
PC$ROLUP	roll up a partial line
PC$ROLDWN	roll down a partial line
PC$NEGLF	output negative line feed
PC$SPLPT0	special print position 0
PC$SPLPT1	special print position 1
PC$RIBSC1	print in secondary ribbon color 1
PC$RIBSC2	print in secondary ribbon color 2
PC$RIBSC3	print in secondary ribbon color 3
PC$RIBPRM	print in primary ribbon color
PC$FEDTR1	select feeder tray 1
PC$FEDTR2	select feeder tray 2
PC$FEDTR3	select feeder tray 3
PC$FEDTR4	select feeder tray 4
PC$FEDTGL	select tray 1 and tray 2 thereafter
PC$FEDEJT	select feeder eject
PC$USR1	user function 1
PC$USR2	user function 2
PC$USR3	user function 3
PC$USR4	user function 4
PC$RIBSC4	print in secondary ribbon color 4
PC$RIBSC5	print in secondary ribbon color 5
PC$RIBSC6	print in secondary ribbon color 6
PC$RIBSC7	print in secondary ribbon color 7

2.4.5 P.SPL - Output Special Control String

Inputs: D1 - control string
D2 - byte to output

This function existed in the traditional printer driver standard to facilitate the direct output of printer-specific codes by earlier releases of AlphaWRITE.

This call is rendered obsolete by AlphaWRITE version 2.0 and later, and the extended printer driver standard, and should no longer be used.

2.4.6 P.CRLF - MOVE TO NEXT LINE

This function is responsible for moving the print cursor—or printhead—to the next line. The `P.CRLF` function must take the current line spacing (as set by the `P.SLSP` function) into account. If the printer supports leading, the `P.LEAD` setting must also be taken into account.

The points-oriented `P.MOVE` command should be used in preference to the `P.CRLF` command wherever possible. The `P.MOVE` command is more flexible from an applications programming viewpoint, at the expense of adding complexity to the `P.MOVE` command.

2.4.7 P.TOF - MOVE TO TOP OF FORM (START NEXT PAGE)

This call moves to the start of the next page. On older and simpler printers, the `PDTOF` routine simply outputs a form feed to the output file:

```
PDTOF: MOVB      #$FF,D1
       FILOTB    PTDDB(A3)
       RTN
```

2.4.8 P.PSON - ENABLE PROPORTIONAL SPACING

`P.SON` is called by an application program that is about to send a line of justified text to the output file. This warns the printer driver that printhead movements may be required after each space character is output. The term "proportional spacing" in this context has nothing to do with proportional fonts (vs. monospaced fonts). It refers to spreading out space evenly along the line—a facility called "microjustification" on some older dot matrix printers.

The additional amount of points to move (that over and above that required to print a space character) is placed in the `WRDSPC(A3)` variable in 6-byte Alpha Micro floating point format. This number represents the amount of inter-character spacing (in points) divided by the number of spaces on the line. It is up to the printer driver's `P.CHR` routine to take this amount into account when printing a space.

When the line has been sent, a `P.PSOF` call must be issued to terminate this spacing mode.

These functions worked differently under the old printer driver standard, mostly because of differences in units of measurement. Old printer drivers that implemented proportional spacing will probably not work as-is with AlphaWRITE version 2.0 and later.

2.4.9 P.PSOF - DISABLE PROPORTIONAL SPACING

The `P.PSOF` routine turns off the spacing mode explained in the `P.PSON` section, and is always called after a justified line has been output. This routine should clear `WRDSPC(A3)`.

2.4.10 P.UNDR - TOGGLE UNDERSCORE

The `P.UNDR` routine toggles underscored printing.

2.4.11 P.BOLD - TOGGLE BOLD

The `P.BOLD` routine toggles bold printing.

2.4.12 P.STRK - TOGGLE STRIKEOUT

Inputs: D1 - character to strikeout with

The `P.STRK` routine toggles strikeout. The character to strikeout with is passed in register D1. The printer driver is responsible for remembering the character and its `P.CHR`, `P.OTSC` and `P.OTEC` routines are responsible for over-typing each character with the strikeout character until the next `P.STRK` call is made.

2.4.13 P.BAR - TOGGLE OVERBAR

The `P.BAR` routine toggles overbar printing.

2.4.14 P.SLPI - SET VERTICAL MOTION INDEX

Inputs: D2 - VMI in lines per inch.

The `P.SLPI` call sets the printer's lines per inch, or vertical motion index (VMI). Register D2 contains the number of lines per inch required.

Points-oriented extended printer drivers should implement the `P.LEAD` function, and points-oriented applications should not call `P.SLPI`.

2.4.15 P.SHMI - SET HORIZONTAL MOTION INDEX

Inputs: D2 - HMI in characters per inch.

The `P.SHMI` call sets the horizontal motion index. For older printers with monospaced characters, this call should be implemented. Points-oriented devices such as laser printers do not need to implement this call. Points-oriented applications should not call this function; the commands to set font family and font size should be used instead.

The horizontal motion index is passed in register D2.

2.4.16 P.SCPI - SET CHARACTERS PER INCH (PITCH)

Inputs: D2 - pitch

The `P.SCPI` call sets the pitch in characters per inch. It is primarily used in older printers such as daisywheel printers. Points-oriented devices do not need to implement this call. Points-oriented applications should not call this function; the commands to set font family and font size should be used instead.

The number of characters per inch is passed in register D2.

Pitch can be controlled in this manner, or by the driver's font table. Select the most natural method for the type of printer in use.

2.4.17 P.STM - SET TOP MARGIN

Inputs: D2 - top margin in lines

This call sets the top margin in lines. It is the traditional method of specifying the top margin. D2 contains the number of lines.

Points-oriented devices such as laser printers should implement the `P.TMAR` call, which has point-resolution instead of line-resolution. Points-oriented applications should also call `P.TMAR`, not `P.STM`.

2.4.18 P.MTM - MOVE TO TOP MARGIN

This call moves to the top margin. On dot matrix printers, this suggests forward line movement until the proper position is reached. Laser printers can perform a direct print cursor address to the top margin.

2.4.19 P.SLPO - SET LEFT PAPER offset

Inputs: D2 - left margin in characters

The `P.SLPO` call sets the left paper offset in characters. This is the traditional method of specifying the left margin. D2 contains the number of 1/10 inch units.

Points-oriented devices, such as laser printers, should implement the `P.LEFT` call, which sets the left margin with point accuracy. Points-oriented applications should not call this function, and use the `P.LEFT` function instead.

2.4.20 P.MLPO - MOVE TO LEFT PAPER OFFSET

The `P.MLPO` call moves the print cursor to the left paper offset position, and is called before sending any character data for the current line.

Points-oriented applications should use the `P.MOVE` call to move the print cursor to the left paper offset.

2.4.21 P.SLPP - SET LINES PER PAGE

Inputs: D2 - lines

The `P.SLPP` call sets the page size in lines. It is the traditional method for specifying the size of the page. D2 contains the number of lines per page.

Points-oriented devices such as laser printers should implement the `P.PSIZ` call, which sets the page size in points. Similarly, points-oriented application programs should invoke the `P.PSIZ` call, not the `P.SLPP` call.

2.4.22 P.SLSP - SET LINE SPACING

Inputs: D2 - number of half lines

The `P.SLSP` call sets line spacing in units of half lines. Register D2 contains the number of half lines.

D2	MEANS
2	Single spacing
3	Line and a half spacing
4	Double spacing
6	Triple spacing

The `P.MNLN` function must take this setting into account when moving to a new line.

2.4.23 P.OVRP - OVERPRINT

The `P.OVRP` call is made to overprint the previous character.

For point-addressable devices such as laser printers, the width of the last character printed is stored in the variable `CWIDTH(A3)` in Alpha Micro 6-byte floating point format. The application program is responsible for storing that value in `CWIDTH(A3)` before calling this function.

2.4.24 P.LF - OUTPUT MULTIPLE LINE FEEDS

Inputs: D2 - number of lines to advance

The `P.LF` call advances zero or more lines. Register D2 contains the number of lines to advance.

Make sure that the printer driver checks for a value of zero. If it does not, and traditional methods of looping are used, the printer driver will output over two billion line feeds when it is passed a value of zero in D2.

Points-oriented drivers and applications programs should use the `P.MOVE` function in preference to this function.

2.4.25 P.DBL - TOGGLE DOUBLE UNDERSCORE

The `P.DBL` function toggles double underscore.

Printers without a double underscore capability should simulate the function or use standard underscore.

2.4.26 P.FONT - SELECT FONT FAMILY

Inputs: D1 - font family name

The `P.FONT` function selects a typeface family. The resultant font rendered is the specified typeface family in the last font size specified. Font sizes are changed with the `P.FSIZ` call.

The font family is specified as a single letter or digit in the low order byte of register D1. Font family names supported by AlphaWRITE versions 2.0 and later are shown below.

FONT FAMILY SPEC	DESCRIPTION
A	Avant-Garde
B	Bookman
C	Courier
D	Zapf Dingbats
E	Prestige Elite
F	Broadway
G	Garamond
H	Helvetica
I	Helvetica-Narrow
J	Cooper
K	Presentations
L	Letter Gothic

FONT FAMILY SPEC	DESCRIPTION
M	Coronet
N	New Century Schoolbook
O	Optima
P	Palatino
Q	unassigned
R	Bodoni
S	Symbol
T	Times
U	University
V	Line Printer
W	unassigned
X	unassigned
Y	unassigned
Z	Zapf Chancery

2.4.27 P.ECHR - OUTPUT EXTENDED CHARACTERS

This call is part of the traditional standard, and becomes obsolete with the new extended standard. It was used to output foreign language characters.

2.4.28 P.PSIZ- SET PAGE SIZE IN POINTS

Inputs: D2 - page size in points

The `P.PSIZ` call sets the page length in points. This allows far greater resolution than the traditional `P.SLPP` call. The page length in points is specified in register D2.

This is an extended printer driver function.

2.4.29 P.TMAR - SET TOP MARGIN IN POINTS

Inputs: D2 - top margin in points

The `P.TMAR` call sets the top margin in points. This allows far greater resolution than the traditional `P.STM` call. The top margin in points is specified in register D2.

`P.TMAR` is an extended printer driver function.

2.4.30 P.LEFT - SET LEFT MARGIN IN POINTS

Inputs: D2 - left margin in points

The `P.LEFT` call sets the left margin in points. This allows far greater resolution than the traditional `P.SLPO` call. The left margin in points is specified in register D2.

This is an extended printer driver function.

2.4.31 P.LEAD - SET LEADING

Inputs: D2 - leading in points

The `P.LEAD` call sets the baseline-to-baseline vertical distance (leading). The vertical distance, in points, is passed in register D2.

This is an extended printer driver function for printers with leading control.

2.4.32 P.FNAM - SET FONT BY NAME

Inputs: A6 - address of ASCIZ font name

The `P.FNAM` call is for future expansion, when the number of available fonts exceeds that which can be represented by single-letter names. Future versions of AlphaWRITE will allow long font names, and this call will be used to perform font selection.

The font name is passed as an ASCIZ string, indexed by A6.

This is an extended printer driver function.

2.4.33 P.FSIZ - SET FONT SIZE

Inputs: D2 - font size in points

The `P.FSIZ` call sets the font size. The size specified and the last typeface designated by a `P.FONT` call are used to render a font.

The font size is always specified as an integer number of points.

AlphaWRITE 2.0 and later pre-scan the printer driver's font table and only request point sizes that are known to exist. Since not all applications that call printer drivers may perform this courtesy, the printer driver should do something reasonable when asked for a point size that does not exist. The usual handling is to select the nearest available point size to the one requested.

This is an extended printer driver function.

2.4.34 P.ITAL - TOGGLE ITALICIZED PRINTING

This function toggles italicized printing.

This is an extended printer driver function.

2.4.35 P.SCRN - TOGGLE SCREENED PRINTING

This function toggles screened printing and is an extended printer driver function.

This is an extended printer driver function.

2.4.36 P.REV - TOGGLE REVERSED PRINTING

This function toggles reversed printing.

This is an extended printer driver function.

2.4.37 P.FATR - SET FONT ATTRIBUTES DIRECTLY

Inputs: D2 - attribute flags

This call allows the application to directly set the various print effects without regard to their previous condition. The printer driver must enable or disable print effects until the requested attribute mask is satisfied.

The attribute flags are:

FLAG	ATTRIBUTE
PA$BLD	bold
PA$UND	underscore
PA$ITA	italic
PA$SCR	screened
PA$REV	reverse
PA$DBL	double underscore
PA$BAR	overbar
PA$GRY	gray

This is an extended printer driver function.

2.4.38 P.MCHR - MOVE TO CHARACTER POSITION

Inputs: D2 - character position

The `P.MCHR` call moves to the specified character position. The character position is passed in register D2.

This is an extended printer driver function.

2.4.39 P.MLIN - MOVE TO LINE POSITION IN D2.

Inputs: D2 - line position

The `P.MLIN` call moves to the specified line position. The line position is passed in register D2.

`P.MLIN` is an extended printer driver function.

2.4.40 P.MOVE - DIRECT MOVE TO COORDINATE LOCATION ON PAGE

Inputs: D1 - horizontal points coordinate
D2 - vertical points coordinate

The `P.MOVE` function moves to a position on the page. The horizontal coordinate is passed in D1, and the vertical coordinate is passed in D2. Both values are specified in points.

The internal coordinate system used by calling applications and printer drivers considers the top left corner of the page to be `(0,0)`. Note that this is different from the PostScript coordinate system, which considers `(0,0)` to be the bottom left corner of the page.

This is an extended printer driver function.

2.4.41 P.SAVE - SAVE PRINT STATE

The `P.SAVE` call saves the current print context for later restoration.

This is an extended printer driver function.

2.4.42 P.REST - RESTORE PRINT STATE

The `P.REST` call restores the last print context saved with a `P.SAVE` call.

This is an extended printer driver function.

2.4.43 P.FTBL - RETURN INDEX TO FONT TABLE

Outputs: A6 - address of font table

This call returns, in register A6, the address of the printer driver's font table.

This is an extended printer driver function.

2.4.44 P.BYTE - DIRECTLY OUTPUT BYTE

Inputs: D1 - byte to output

The P.BYTE call is used during image processing to output an encoded image byte to the printer. This is the only purpose for which this call should be used.

This is an extended printer driver function.

2.4.45 P.SPOR - SET PAGE ORIENTATION

Inputs: D1 - orientation

The P.SPOR call sets the printer's orientation to portrait or landscape. If the low order byte of D1 contains "P", portrait orientation is selected. If the low order byte of D1 contains "L", landscape orientation is selected. Any other value should be ignored by the printer driver.

This is an extended printer driver function.

2.4.46 P.GRAY - SET GRAY SHADING

Inputs: D2 - shading

The P.GRAY call sets the level of gray shading for subsequent output of text and graphics. Register D2 contains a number between 0 and 100 decimal which represents the degree of blackness. A value of 0 should yield white on white (invisible), and a value of 100 should yield true black on white.

This is an extended printer driver function.

2.4.47 P.GRAF - PERFORMS GRAPHICS FUNCTION

Inputs: D2 - graphics function code

The P.GRAF function implements graphics functions. Register D2 contains a function code as shown in the following table.

CODE	FUNCTION
PG$BL	Begin line
PG$EL	End line
PG$BB	Begin box
PG$EB	End box
PG$BR	Begin rounded box
PG$ER	End rounded box
PG$BG	Begin gray box
PG$EG	End gray box

The graphic is drawn to intersect with the bottom left-hand corner of the character's bounding box.

This is an extended printer driver function.

2.4.48 P.FSUB - PERFORM FONT SUBSTITUTION

Inputs: D1 - font family character
Outputs: D1 - font family character

This call substitutes the font in D1 with one the printer actually supports. This may or may not leave the character in the low order byte of D1 unchanged.

AlphaWRITE calls this function when printing if the font substitution feature is enabled.

This is an extended printer driver function.

2.4.49 P.TRSC - TRANSLATE SPECIAL CHARACTER

Inputs: D1 - special character code to translate
Outputs: D1 - translated character

AlphaWRITE calls this function to determine the width of a special character, such as a bullet, that does not have an ISO Latin-1 equivalent. The printer driver function should return in D1 a character code that will yield the proper character width when looked up in a font width table.

This is an extended printer driver function.

2.4.50 P.TREC - TRANSLATE EXTENDED CHARACTER

Inputs: D1 - extended character code
Outputs: D1 - translated character

AlphaWRITE calls this function to determine the width of an extended character that does not have an ASCII equivalent. The printer driver function should return in D1 a character code that will yield the proper character width when looked up in a font width table.

This is an extended printer driver function.

2.4.51 P.OTSC - OUTPUT SPECIAL CHARACTER

Inputs: D1 - special character code to output

`P.OTSC` is called by an application instead of `P.CHR` to output a special character. The routine is responsible for implementing the special character's output. This is often as simple as changing the character code in D1 and jumping to the `P.CHR` routine. Postscript printer drivers may, on the other hand, issue Postscript command to draw the necessary character "from scratch". The character(s) should be output to the output file through its DDB at `PTDDB(A3)`.

This is an extended printer driver function.

2.4.52 P.OTEC - OUTPUT EXTENDED CHARACTER

Inputs: D1 - extended character code to output

`P.OTEC` is called by an application instead of `P.CHR` to output an extended character. The routine is responsible for implementing the extended character's output. This is often as simple as changing the character code in D1 and jumping to the `P.CHR` routine. Postscript printer drivers may, on the other hand, issue Postscript command to draw the necessary character "from scratch". The character(s) should be output to the output file through its DDB located at `PTDDB(A3)`.

This is an extended printer driver function.

2.4.53 P.SETL - SET LINE WEIGHT

Inputs: D2 - line weight in points
`WRDSPC(A3)` - line weight in points (6-byte Alpha Micro floating point number.)

When the printer driver initializes it should set a line weight of 1 point. Using the `WRDSPC(A3)` value permits line weights that are non-integer numbers, such as 2.5 points.

`P.SETL` is an extended printer driver function.

2.4.54 P.POVR - OUTPUT PAGE OVERLAY

Inputs: A6 - address of ASCIZ overlay text

The `P.POVR` outputs a page overlay. A page overlay is diagonal gray text that overlays the normal text of a page. Common uses of page overlay include "confidential" and "preliminary" documents.

The overlay text is centered diagonally on an 8.5" x 11" page at an inclination of 53 degrees. This function must restore the original print state. Specifically, the print cursor location and typeface selection must not be altered by this routine.

Register A6 contains the address of an `ASCIZ` string that will be displayed on the page.

This is an extended printer driver function.

2.4.55 P.IMON - IMAGE PROCESSING ON

Inputs: D2 - printed height in points

The `P.IMON` call begins the output of an image to the printer. The `P.BYTE` call should then be called successively by the application, and finally a `P.IMOF` call should be made

to terminate processing of the image.

Some printers can scale images. A desired print height, in points, is passed in register D2. The printer driver may use or discard this value as appropriate.

AlphaWRITE positions the print cursor by taking the previous location on the page and adding the vertical printed height to that position. This allows image files to presume that they are starting with the lower left corner of their bounding box as the current point.

The `P.IMON` call must preserve the print state so that the eventual call to `P.IMOF` will restore the original page position and typeface selection.

This is an extended printer driver function.

2.4.56 P.IMOF - IMAGE PROCESSING OFF

The `P.IMOF` call turns off image processing. This call restores the original print cursor position, prior to the last `P.IMON` call.

This is an extended printer driver function.

2.4.57 P.KEY - TOGGLE KEYCAP PRINTING

The `P.KEY` function toggles the keycap print feature.

This is an extended printer driver function.

2.4.58 P.CART - SET CARTRIDGE/PRINTWHEEL/THIMBLE

Inputs: D2 - setting

The `P.CART` routine is used to specify cartridge, printwheel, or thimble changes. The specifics of register D2 are printer-dependent.

2.4.59 P.KERN - SET TRACK KERNING

Inputs: D2 - kerning amount

The P.KERN function sets track kerning based on the code value in D2. The `P.CHR` function must move left by a corresponding amount after printing each character.

The calling application passes one of the values shown below in register D2 to specify the type of track kerning that should be used.

Value	Meaning
0	no track kerning
2	light kerning: decrease spacing by 5%
4	medium kerning: decrease spacing by 10%
6	heavy kerning: decrease spacing by 15%

A value of 0 turns off track kerning.

The printer driver does not have access to the font width tables. Therefore it is the responsibility of the applications program to do the line length calculations and to tell the driver by how much to kern each character. As each character has a different width, the application number must put the appropriate kerning amount as an Alpha Micro 6-byte floating-point format value (in points) in `PDVARG(A3)` in the communications area. The printer driver's `P.CHR` routine should check if kerning is enabled and, if so, move the printhead `PDVARG(A3)` points after printing the character.

2.4.60 P.SPAG - SET PAGE PRINTING

The `P.SPAG` call is reserved for future use.

2.4.61 P.OUTL - TOGGLE OUTLINE FONTS

`P.OUTL` toggles the outline print effect.

This is an extended printer driver function.

2.4.62 P.CBAR - TOGGLE CHANGE BARS

The `P.CBAR` call is reserved for future use.

2.4.63 P.PCH - RETURN PRINTER CHARACTERISTICS

`P.PCH` returns the following detailed printer information into a 256. byte area set up by the applications program and indexed by register A6.

NAME	DESCRIPTION	SIZE
PC.MAK	Make of printer (ASCIZ).	20. bytes
PC.MOD	Model of printer (ASCIZ).	20. bytes
PC.LNG	Name of page description language (ASCIZ)	20. bytes
PC.EXT	Valid image file extensions	8. RAD50 words
PC.XXX	Reserved for future expansion	180. bytes

2.4.64 P.DDPE - TOGGLE DEVICE-DEPENDENT PRINT EFFECT

The `P.DDPE` call is reserved for future use.

2.5 FONT TABLE

Printer drivers with support for multiple fonts must contain a font table. The `P.FTBL` function returns an index to this table. Both the printer driver and AlphaWRITE scan this table to make font selection decisions.

2.5.1 FONT TABLE STRUCTURE

The font table contains one or more font entries. The format of a font entry is detailed in section 2.5.2. A word of zero marks the end of the font table.

Depending on the type of printer, a single font entry may define an entire typeface, or just one font size of a family. When the printer has scalable fonts, such as a PostScript printer, just one entry per typeface style is required. When the printer supports discrete fonts in a family, such as the HP LaserJet II, a separate entry for each font size is required.

When a typeface is operated on by bold, italic, or both, it is considered a separate font, and requires a separate entry in the font table.

2.5.2 FONT ENTRY STRUCTURE

A font entry is a fixed-size structure composed as follows:

FLAG	MEANING	AREA SIZE
FO.LOG	Logical name of font	30. bytes
FO.CAR	Cartridges supporting font	10. bytes
FO.PHY	Physical name of font	40. bytes
FO.FLG	Font flags	4 bytes
FO.TBL	Width table name	4 bytes
FO.SIZ	Type size	2 bytes
FO.LED	Leading	2 bytes
FO.CPI	Characters per inch	2 bytes

Each field of the font entry is defined below.

2.5.2.1 FO.LOG - Logical Name Of Font

The `FO.LOG` field contains the logical font name as a string padded with nulls to a length of 30. bytes. Currently, AlphaWRITE supports single letter or single digit font names only.

2.5.2.2 FO.CAR - Cartridges Supporting Fonts

The `FO.CAR` field contains the single-letter designations of font cartridges that support this font. For printers without selectable font cartridges, this field may be left blank. The list of ASCII letters should be padded with nulls to a length of 10. bytes.

2.5.2.3 FO.PHY - Physical Name Of Font

The `FO.PHY` field contains the physical string or control sequences required to select the font. The printer driver may or may not use this field at its discretion. The calling application is prohibited from making use of this field.

2.5.2.4 FO.FLG - Font Flags

The `FO.FLG` longword field contains flag bits describing attributes of the font.

FLAG	MEANING
FO$PRO	Proportionally-spaced font
FO$KRN	Kerning information exists for font
FO$BLD	Bold rendition of a typeface
FO$OBL	Italic rendition of a typeface
FO$SCL	Font is scalable to any size
FO$POR	Font available in portrait orientation
FO$LAN	Font available in landscape orientation
FO$RES	This is a resident font

2.5.2.5 FO.TBL - Name Of Font Width Table

The `FO.TBL` longword field contains the RAD50 name of the font's font width table (`.FWT`) file. This field should be set to zero if there is no font width table available. Font width tables are not required for monospaced fonts.

2.5.2.6 FO.SIZ - Font Size In Points

The `FO.SIZ` field contains a binary word value which is the font size in points. If the `FO$SCL` flag is set, indicating a scalable font, this field should contain zero.

2.5.2.7 FO.LED - Leading In Points

The FO.LED field contains a binary word value which is the leading (vertical spacing between baselines) in points. This should only be set if the printer does not have its own leading control. If the printer has leading control, set this field to zero.

A value of `-1` will use the document's current leading setting.

2.5.2.8 FO.CPI - Characters Per Inch

The `FO.CPI` field contains a binary word value which describes the characters per inch (pitch) for monospaced fonts. The true pitch for the fonts is multiplied by 100 and integerized. Thus, a `FO.CPI` value of 1000. indicates 10 pitch, and 1666. indicates 16.66 characters per inch.

For proportional fonts this field should be set to zero. A value of `-1` will use the document's current pitch setting.

2.6 FONT WIDTH TABLES

Font width tables contain character spacing information. AlphaWRITE version 2.0 and later, and other applications, consult them during line fill and justification to provide a typeset-like appearance.

AlphaWRITE is capable of processing two varieties of font width tables:

- AFM (Adobe Font Metric) format
- Centipoints format

All font width tables have an extension of `.FWT` and reside in the `WRT:` or the `:FWT` account. All `.FWT` files begin with a flags longword indicating their type.

FLAG	MEANING
FT$AFM	Adobe Font Metric-derived format
FT$PTS	Centipoints format

2.6.1 AFM FORMAT FONT WIDTH TABLES

An AFM Font Width Format is derived from Adobe's Font Metric (AFM) font description format. The font width table consists of a flags longword, a character spacing table, and a pair kerning table.

2.6.1.1 Flags Longword

The flags longword sets the `FT$AFM` flag to indicate that this is an AFM type font width table.

2.6.1.2 Character Spacing Table

The character spacing table consists of 256 entries, one for each character in the ISO Latin- 1 character set. Each entry consists of seven words, as follows.

WORD	MEANING
Word 1	Horizontal width adjustment
Word 2	Horizontal adjustment
Word 3	Vertical adjustment
Word 4	Bounding box lower left x coordinate
Word 5	Bounding box lower left y coordinate
Word 6	Bounding box upper right x coordinate
Word 7	Bounding box upper right y coordinate

AFM format font width tables are for scalable fonts. The values are in units of 1/1000th points and are for 1-point characters. The calling application must scale the values to the current font size.

AlphaWRITE currently uses only the horizontal width adjustment. The other fields are for future enhancements.

2.6.1.3 Pair Kerning Table

The pair kerning table consists zero or more pair kerning entries, terminated by a word of zero. Each entry is structured as follows.

WORD	MEANING
Word 1	First character in kerning pair
Word 2	Second character in kerning pair
Word 3	Horizontal adjustment
Word 4	Vertical adjustment

The values are in units of 1/1000th points and are for 1-point characters. The calling application must scale the values to the current font size.

AlphaWRITE uses only the horizontal adjustment. The vertical adjustment field is for future enhancements.

2.6.2 CENTIPOINTS-FORMAT FONT WIDTH TABLES

Centipoints-format font width tables are used for printers with discrete font sizes, such as the HP LaserJet II. After the flags longword is a character spacing table. The table consists of 256 entries in the ISO Latin-1 character set.

Each entry in the character spacing table is a word describing a character width in integer centipoints (7200 centipoints = 72 points = 1 inch).

Currently, neither the printer driver standards nor AlphaWRITE support kerning tables for centipoints-format font width tables.

CHAPTER THREE
COMMAND LINE PROCESSING

WHAT IS COMMAND LINE PROCESSING?

Software developers can access AlphaWRITE's powerful word processing capabilities from their own software applications. Applications can access AlphaWRITE's functions via the WRITE command line.

Your application can request specific AlphaWRITE commands directly, or process a script file containing a series of more complicated instructions. This capability can let your software program take advantage of AlphaWRITE's word processing features and/or automate repetitive AlphaWRITE operations.

HOW COMMAND LINE PROCESSING WORKS

Implementation of AlphaWRITE command line processing follows the same theories as AlphaWRITE's Glossary feature. When you create an AlphaWRITE glossary entry to perform operations—rather than to hold document text—the entry must contain complete step-by-step instructions for every action you want performed. You supply the text and the universal key sequence instructions for any keys to be pressed via the glossary entry.

For example, a glossary entry to display the Editing Menu from document text display and then begin the **SEARCH** command looks like this:

```
^[^[search^M
```

Where `^[^[` represents the universal key sequence for pressing [MENU], `SEARCH` is the command to use, and ^M represents the universal key sequence for [RETURN].

When a software program uses AlphaWRITE's command line option to access the word processor, it must provide complete step-by-step instructions for each operation, just like the glossary entry. However, you can combine command line processing with user input. For example, you can begin a **PRINT** command, then allow the user to answer all, or only some, of the questions **PRINT** asks.

COMMAND LINE FORMAT

From AMOS command level, the format for using command line processing is:

```
WRITE {option option option,} {command command ...}
```

Where `{option}` represents any one or more of the options shown in the next section of this document and the `{command}` are the instructions for the operation you want AlphaWRITE to perform.

Spaces on the command line are ignored. You can include as many or as few spaces as you want to make the line easy to enter or understand.

COMMAND LINE OPTIONS

The WRITE command options are shown in the following table.

OPTION	DESCRIPTION
/P	Displays AlphaWRITE's Product Installation Code (PIC). Does not invoke word processing operations. For example: **WRITE/P** RETURN
Filename	Causes the WRITE command to immediately edit the AlphaWRITE document identified by filename. For example: **WRITE MYDOC** RETURN
/C	Invokes command line processing so AlphaWRITE identifies the following characters as commands and not a filename to edit. For example: **WRITE/C /D, PRINT** RETURN This example causes screen display to appear and begins the print command. When you are through using the command, this example automatically returns to AMOS command level. (Note the comma following /D to signal PRINT as an AlphaWRITE command.) When you use the /D, /NOD, /S, or /NOS command option, you can omit /C. These options also identify the characters following them as commands.
/D	Causes AlphaWRITE menus and other screens to appear on the user's terminal.

OPTION	DESCRIPTION
/NOD	Causes AlphaWRITE to suppress display of its menus and other screens while it processes the operation requests. Default. Comma must appear following D.
/S	Causes user to remain in AlphaWRITE, in manual mode, after all command line processing is complete. Comma must appear following the S.
/NOS	Determines how the user exits AlphaWRITE. Default. If command line processing stops at the Main Menu, when the user presses RETURN they will exit from AlphaWRITE. If command line processing finishes elsewhere in AlphaWRITE, the user remains in AlphaWRITE. When any feature which normally returns to the Main Menu is used, /NOS causes exit AlphaWRITE.
/F:Filespec	Causes AlphaWRITE to process the script file identified by filespec. Especially useful when you want to perform many word processing operations. There is a detailed discussion of using script files later in this chapter.

COMMAND LINE SYNTAX

The following sections discuss some rules you must follow and features you can use when using command line processing. They include:

- When commas are required on the command line.
- AlphaWRITE's type-ahead feature in command line processing.
- Using the caret symbol (^) or apostrophe (') as a text character.
- Interrupting or cancelling command line processing.
- How command line processing handles errors during its operation.

Commas

A comma is required only to separate the list of options from a list of commands. If there are no commands, a trailing comma is not required.

For example, this command line erases the file named MYDOC. It displays AlphaWRITE's screens, and lets the user remain in AlphaWRITE in manual mode when it finishes:

```
WRITE/D/S, erase mydoc^M^[^[ RETURN
```

Type-Ahead

You can use AlphaWRITE's type-ahead feature with command line processing, as shown in the example above: it supplies the document name to erase immediately after the command "erase" without an intervening RETURN. As with AlphaWRITE's glossary feature, you must supply complete instructions such as, pressing RETURN (^M) and MENU (^[^[).

Using Caret (^) and Apostrophe (') as Text Characters

If you must pass the caret symbol (^) as a character and not as part of a key sequence, use the key sequence "apostrophe caret". Like this:

```
'^
```

The apostrophe causes the WRITE command to interpret the ^ symbol as text and not the beginning of an escape-key or control-key sequence. Any character after the apostrophe is considered text.

Because of this special use of the apostrophe, you must use "apostrophe apostrophe" to include apostrophe as text:

```
''
```

Cancel Command Processing

You can cancel command line processing by pressing CTRL-C. The command line processing stops with the last command performed; the rest of the command line is ignored. You are left at the current position in AlphaWRITE, in manual mode.

If AlphaWRITE is currently processing a command from the command line—for example, if you cancel command processing during a **PAGINATE**—the process does not stop. After pressing CTRL-C to stop the command line, you can then press MENU to stop the **PAGINATE**.

Error Handling

Errors do not halt command line processing. Indeed, some command line scripts are defined to exhibit errors. The only way to halt command line processing is by pressing CTRL-C.

COMMAND LINE PROCESSING SAMPLE

This section shows an example of using command line processing. The command below allows screen display during the **PRINT** command on the document named RESULT:

`WRITE/C/D/S, print result^M` [RETURN]

The parts of the command line are:

- /C specifies an AlphaWRITE command is being requested rather than a filename for a script file.
- /D requests AlphaWRITE to display screens while the command processes.
- /S, requests that the user remain in AlphaWRITE when command line processing is complete—unless you explicitly request exit—and the comma signals the next text is an AlphaWRITE command.
- `print result^M` is the command name **PRINT**, document name RESULT, and [RETURN].

You could leave out /C without altering the meaning of this command, since /D and /S also indicate an AlphaWRITE command will follow.

If you omit /S from the command line, AlphaWRITE defaults to /NOS and you will remain in AlphaWRITE, in the midst of the **PRINT** command, so you can respond to **PRINT**'s questions. When you press [MENU] to return to the Main Menu, it will exit AlphaWRITE.

USING SCRIPT FILES

With the /F: option, you can use AlphaWRITE 's command line processing to perform a series of commands stored in an AlphaWRITE document or an AlphaVUE text file. As stated earlier in this chapter, the format for using a script file is:

`WRITE /F:Filespec` [RETURN]

The default extension for the file specification is .WRT.

The file you specify should contain a series of AlphaWRITE commands and instructions in the same format as they would be in a WRITE command line or a glossary entry.

When processing a script file, AlphaWRITE considers all printable characters as part of the series of instructions. This means:

- All blank lines and paragraph markers (in AlphaWRITE documents) are ignored.
- Since spaces are considered printable characters, spaces are **not** ignored. Be careful not to include extra spaces which might affect the AlphaWRITE operation.

- You can include comments in your script file. To signal a comment, the first two characters on the line must be "colon slash" (**`:/`**), like so:

```
:/This is comment text
```

In addition, script files follow the type-ahead, caret, apostrophe, interrupt, and error processing command line processing rules described earlier.

Chaining Script Files

You can invoke another script file from within a script file. The nested script file specification must appear on a line by itself in this format:

```
/F:FILESPEC
```

The file you call must follow all the rules for a script file, as described above. The default extension is .WRT.

When you call a second script file in this manner, control passes to the new file, and any following instructions in the original script file are ignored. For example: You have two script files, one named PARENT.WRT and one named CHILD.WRT. You invoke PARENT.WRT by typing:

`WRITE/C /D,/F:PARENT.WRT` RETURN

The PARENT.WRT script file contains commands to edit the document named MYDOC, go to the end of the document, then call the file named CHILD.WRT:

```
EDIT MYDOC^M^E
/F:CHILD.WRT
```

If there are any commands following /F:CHILD.WRT in PARENT.WRT they will be ignored because control has been transferred to CHILD.WRT. Control will never revert to the PARENT.WRT script file.

Pausing Script Files

Two special commands lets you pause a script file during processing:

- Pause (:K). If a line in a script file contains only :K, AlphaWRITE will wait for a single keystroke. This keystroke is passed to AlphaWRITE, then the script file resumes with the next instruction.

- Wait (:W). If a line in a script file contains only `:W` n where n is the number of clock ticks to wait, AlphaWRITE will delay further processing of the script file for that period of time.

You can also pause a script file from the keyboard during processing. Press CTRL-S to halt the processing, and CTRL-Q to resume it.

Using Colons

To include a colon in the first column of your script file and not signal the beginning of a special instruction, use two colons (`::`).

File Example to Use with /F: Option

The sample below represents the contents of a file named FILEIT.WRT. FILEIT creates a directory list of all AlphaWRITE documents in this account in a disk file named DIRECT.LST, creates a document named CABNET, changes the right margin setting, pastes DIRECT.LST into CABNET, finishes from the document and exits AlphaWRITE.

A brief explanation appears on the line before each command; the command line itself is in bold type to make it easy to see. The sample assumes you have more than one printer defined and there are no sheet feeders associated with printer 1.

See the *AlphaWRITE Reference Guide* or *AlphaWRITE Terminal Keyboard Commands* for a complete list of universal key sequences for AlphaWRITE features.

```
:/           FILEIT.WRT: Sample Script File
:/
:/ run directory command:
dir^M
:/ request history list:
h^M
:/ yes, display all documents:
^M
:/ yes, print directory list:
y^M
:/ yes, printer number 1:
^M
:/ yes, disk output only:
y^M
:/ press MENU to return to Main Menu:
^[^[
:/ Create document named cabnet, press Return:
create cabnet^M
:/ Accept DEFDOC prototype document:
^M
:/ Type text in Header’s Description field,
:/ press Return:
Document List^M
:/ Press Execute to go to document display:
^[X
:/ Press ESC SPACEBAR to invoke Margin key:
^[ (spacebar)
:/ Go to Ruler’s end + 5 more spaces, change right margin
^N^L^L^L^L^L>
:/ Press Execute to return to document:
^[X
:/ Press Return to go to next line after margin block,:
:/ then Menu key for Editing Menu:
^M^[^[
:/ Paste directory list made in the beginning into this
:/ file, press Return:
paste direct.lst^M
:/ Use finish to exit document, return Main Menu:
^[^[ finish ^M
:/ Quit from AlphaWRITE:
quit^M
```

To execute the instructions shown in this example, the AMOS command would be:

WRITE/F:FILEIT.WRT RETURN

This form of the command would not display the operations contained in the file and would exit AlphaWRITE upon completion—the command line assumes the defaults /NOD and /NOS.

If you prefer, you could see all operations and remain in AlphaWRITE after the sequence of instructions in FILEIT.WRT is finished by using this command format:

WRITE/D/S/F:FILEIT.WRT RETURN

AlphaBASIC PROGRAM

The following is a sample AlphaBASIC program showing how to use the AlphaWRITE command line processing feature from within a program.

```
1000 ! SAMPLE.BAS
1010 !
1020 ! SAMPLE PROGRAM USING AlphaWRITE's COMMAND LINE PROCESSING CAPABILITIES
1030 ! This program is a menu with three of the commands chaining to AlphaWRITE.
1040
1050        PRINT TAB(-1,36);TAB(-1,0);             ! Turn off and clear screen.
1060
1070 ! Print a reverse video banner.
1080
1090        PRINT TAB(11,80);TAB(-1,33);TAB(11,19);TAB(-1,32);
1100        PRINT " Fine Wine Distribution Version 1.0(100) ";TAB(-1,33);
1110
1120        ! Display the menu options.
1130
1140        PRINT TAB(15,5);"1.  Order Entry"
1150        PRINT TAB(16,5);"2.  Shipping"
1160        PRINT TAB(17,5);"3.  Edit AlphaWRITE file"
1170        PRINT TAB(18,5);"4.  Create AlphaWRITE file"
1180        PRINT TAB(15,40);"5.  Inventory"
1190        PRINT TAB(16,40);"6.  Accounts Receivable"
1200        PRINT TAB(17,40);"7.  AlphaWRITE directory"
1210        PRINT TAB(18,40);"8.  Return to AMOS command level"
1220
1230        ! Get command number from user.
1240
1250 GET'COMMAND:
1260        PRINT TAB(20,5);"Enter command number and press RETURN: ";
1270        PRINT TAB(-1,37);
1280        INPUT A                                 ! Get number.
1290        PRINT TAB(22,1);TAB(-1,9)               ! Erase error message line.
1300
1310 ! Is it a valid command?
1320
1330        IF A<1 OR A>8 THEN GOTO BAD'COMMAND
1340
1350 ! Is the command implemented?
1360
1370        IF A<3 OR A=5 OR A=6 THEN GOTO COMMAND'NOT'IMPLEMENTED
1380
1390 ! Does user need to enter a document name?
1400
1410        IF A=3 OR A=4 THEN GOTO GET'DOCUMENT'NAME
1420
1430        PRINT TAB(-1,36);                       ! Turn off screen.
1440
1450 ! If user entered 7 then chain to the do file DIRECT.DO.
1460 !  The do file DIRECT.DO has the following commands:
1470 !   :R
1480 !   ; AlphaWRITE using the command line file DIR.WRT.
```

```
1490 !   ; The command line file DIR.WRT has the command line :
1500 !   ;   DIR^M^M^M
1510 !   ;   ^M/D,
1520 !   WRITE/C /F:DIR.WRT
1530 !   RUN SAMPLE                                  ; Return to SAMPLE.RUN.
1560
1570        IF A=7 THEN CHAIN "DIRECT.DO"
1580
1590 ! User entered 8. Return to AMOS command level.
1600
1610        PRINT TAB(-1,0);TAB(-1,37);             ! Clear screen
1620        END         ! Exit to AMOS
1630
1640 ! We need to get a document to either EDIT or CREATE with AlphaWRITE.
1650
1660 GET'DOCUMENT'NAME:
1670
1680        PRINT TAB(20,1);TAB(-1,9);
1690        PRINT TAB(21,23);"Enter document name: ";
1700        INPUT LINE DOCUMENT$                    ! Get document name.
1710        PRINT TAB(-1,36);                       ! Turn off screen.
1720
1730 ! Create command line file, FNWN.CMD, to either edit or create an
1740 ! AlphaWRITE file.
1750
1760        OPEN #1,"FNWN.CMD",OUTPUT               ! Open FNWN.CMD.
1770
1780 ! First line of command file is EDIT "document name" or
1790 ! CREATE "document name".
1800
1810        IF A=3 THEN PRINT #1,"EDIT "+DOCUMENT$ ELSE PRINT #1,"CREATE "+DOCUMENT$
1820
1830 ! Second line of command line file will turn on the screen
1840 ! and pass a carriage return.
1850
1860        PRINT #1,"^M/D,"                        ! Second line of cmd line file.
1870        CLOSE #1                                ! Close FNWN.CMD.
1880
1890 ! Chain to the do file SAMPLE.DO.
1900 ! The do file SAMPLE.DO has the following commands:
1910 !  :R
1920 !  ; AlphaWRITE command line using command line file FNWN.WRT.
1930 !  WRITE/F:FNWN.CMD
1940 !  RUN SAMPLE                                  ; Return to SAMPLE.RUN.
1950 !
1960
1970        CHAIN "SAMPLE.DO"
1980
1990 ! Let user know the command chosen isn't implemented and ring bell.
2000
2010 COMMAND'NOT'IMPLEMENTED:
2020        PRINT TAB(22,28);"Command not implemented";CHR$(7);
2030        GOTO GET'COMMAND
2040
2050 ! Let user know an unrecognized command was entered and ring bell.
2060
2070 BAD'COMMAND:
2080        PRINT TAB(22,30);"Unrecognized command";CHR$(7);
2090        GOTO GET'COMMAND
```

CHAPTER FOUR
INSTRUCTIONS FOR TRANSLATING MESSAGE FILES

AlphaWRITE uses a simple message file scheme that is easy to modify as necessary for translation or any other reason. This chapter discusses which message file is used for what purpose, the internal format of the file, and how to translate or modify the file for other languages.

MESSAGE FILE NAMES

The chart below gives the names and functions of all message files used by AlphaWRITE. "*lng*" is the message file extension defined in the language definition file that is active for your job. AlphaWRITE's message files are shipped from Alpha Microsystems with the extension .USA.

FILENAME	FUNCTION
WRT400.*lng*	General purpose
WRT401.*lng*	Debug command
WRT402.*lng*	Outline
WRT403.*lng*	Journal recovery
WRT404.*lng*	Merge
WRT405.*lng*	Printing
WRT406.*lng*	Glossary
WRT407.*lng*	Table of Contents
WRT408.*lng*	Maintenance
WRT409.*lng*	Index
WRT410.*lng*	Margin
WRT411.*lng*	Spell
WRT412.*lng*	Paste
WRT413.*lng*	Format
WRT414.*lng*	Error reporting

FILENAME	FUNCTION
WRT415.*lng*	Printer Queue
WRT416.*lng*	Calendar
WRT417.*lng*	Sort
WRT418.*lng*	Mode
WRT419.*lng*	Help
WRT420.*lng*	Header
WRT422.*lng*	Convert
WRT423.*lng*	8-bit characters
WRT424.*lng*	Cut
WRT425.*lng*	Directory

WRT400.*lng* is by far the largest message file in AlphaWRITE.

OVERVIEW OF MESSAGE FILE CONTENTS

Although the format of an AlphaWRITE message file can vary from file to file, the general internal format is as follows:

- Initial Assembler directives.
- A Dispatch Area.
- A Message Area for display-only messages. This area is split into two sections, the Message Dispatch Area and the Message Table Area.
- A Command Area for text strings, such as valid commands for the AlphaWRITE Main or Editing menus. Like the Message Area, this area is split into two sections, the Command Dispatch Area and the Command Table Area.
- A Field Area defining parameters for user input, such as row and column information, such as entering the name of an AlphaWRITE Main or Editing menu command. This area is also split into two sections, the Field Dispatch Area and the Field Table Area.
- Some final Assembler directives at the end of the file.

`WRT400.`*`lng`* contains a number of other sections that control other sections of AlphaWRITE. You will need to treat `WRT400` with special care when you translate it for other languages.

HOW AlphaWRITE PROCESSES A MESSAGE FILE

To understand the format of a message file, it is useful to know how AlphaWRITE uses a message file.

AlphaWRITE uses message files to hold all language-sensitive text and information about user input areas. All message files can be edited using AlphaVUE or AlphaXED.

For example, the text of the AlphaWRITE Main Menu screen is held in one message file, including the screen positions and attributes for each text string, the position and data validation information for command input, and the strings against which to validate the user's input.

When AlphaWRITE needs to display non-document text on the screen, or prompt for and accept user input, it uses the contents of a message file to decide what text to display and how and where it accepts user input.

AlphaWRITE then locates the appropriate message file, and loads it into memory. It can then open the file and process its contents. The format of the message file must be identical to what AlphaWRITE expects to find, or the processing will fail, with some very strange results.

Let's take a specific example of displaying some text, say the fifth message in a message file. AlphaWRITE directs its attention at the Dispatch Area of the file. It knows that the first `OFFSET` statement in this area points to the Message Dispatch Table. Therefore AlphaWRITE reads the first pointer in the Dispatch Area, and follows it down to the start of the Message Dispatch Area.

AlphaWRITE knows that the fifth `OFFSET` statement in this area points to the stream of bytes that define all the data about that fifth message: the text, cursor positioning and screen attribute information. So, it follows that pointer down to the start of the byte stream of the fifth message, and starts processing the byte stream. In the case of a message, AlphaWRITE knows when to terminate processing the rest of the file: when it encounters a byte with a value of zero (null), it stops processing and gets on with its next operation.

The same principle holds for AlphaWRITE's processing of command or field information. From the Dispatch Area, AlphaWRITE follows a particular chain of pointers down to the required text. If any of the pointers point to the wrong place, AlphaWRITE will blindly follow the pointer, regardless of its direction. It is very important to make sure the pointers point at the correct information. Do check these links after you have modified the file!

The individual processing that occurs at the end of the chain of pointers is dependent on AlphaWRITE's internal instructions. Therefore the format of these areas (the Table Areas) may vary from Area to Area.

In addition, `WRT400.`*`lng`* may have other pointers defined in the Dispatch Area. However, the principle of following the pointers is the same—it's just that there are more of them and you need to check carefully if you need to translate text or set up other data information at the end of those chains.

INITIAL ASSEMBLER DIRECTIVES

Assembler directives do not change when the file is translated into another language.

A message file has two Assembler directives at the top of the file, for example:

```
OBJNAM      .USA
COPY        MSGSYM
```

The OBJNAM Assembler directive sets the filename and/or extension of the file that is created when the file is assembled. As explained later in the "Assembling A Message File" section, you need to specify a language-dependent extension for the assembled file, one that matches both the language of the text in the file and the language file extension defined in the matching AMOS language definition file (the .LDF file). Using the format shown—the period followed immediately by the extension of an AMOS file specification— the assembled file will have the same filename as the source file, but will have the extension defined by the OBJNAM statement.

The COPY statement causes the Assembler to pull in the file MSGSYM.M68. This file is required because it contains definitions of various symbols and commands used in message source files.

A message file does not contain version information, which would have been set up by the PHDR macro. Do not include a PHDR macro, or the results of using the assembled message file will be unpredictable.

DISPATCH AREA

The Dispatch Area does not change when the message file is translated into another language.

The Dispatch Area must appear as the first section of executable code in the file, so it must immediately follow the Assembler Directives. It holds two types of statements, OFFINI and OFFSET:

- The OFFINI statement starts off a dispatch area—a series of pointers to other sections of the file. Internally, it initializes a variable that is used by the OFFSET statement. The OFFINI statement must appear as the first statement in the Dispatch Area, and only one OFFINI statement should appear in this section.

- The OFFSET statement takes as its only parameter a label which defines the location of another section of the file. It literally points to that location. Each label shown in an OFFSET statement must appear exactly once later in the message file. The order of the OFFSET statements is important and must not be changed.

Technically, the OFFINI and OFFSET commands are Assembler macros defined in MSGSYM.M68.

An example of the Dispatch Area is shown below. Most message files will have three OFFSET statement in the Dispatch Area—three pointers, one pointing to the Message Area, one to the Command Area and one to the Field Area. The label preceding OFFINI (WRT402:) is optional, but if you use one, be sure that the label reflects the message file's filename.

```
WRT402:  OFFINI
         OFFSET     MSG       ; pointer to message area
         OFFSET     CMD       ; pointer to command/table area
         OFFSET     FLD       ; pointer to field area
```

MESSAGE AREA

The Message Area consists of two parts:

- Message Dispatch Area.
- Message Table Area.

The Message Dispatch Area must appear after the Dispatch Area, and the Message Table Area must appear after the Message Dispatch Area. Conventionally, both sections appear immediately after the Dispatch Area.

Only the Message Table Area needs to be changed when translating the file to a different language. The Message Dispatch Area remains the same.

The Message Dispatch Area

The format of the Message Dispatch Area is similar to that of the Dispatch Area. The `OFFINI` and `OFFSET` statements are used to initialize the area and define pointers to the location of the individual messages. The `OFFINI` statement must be preceded by a label. That label must be the same label as used in the `OFFSET` command in the Dispatch Area that pointed to the Message Area. As an example:

```
MSG:     OFFINI
         OFFSET     0$    ; M$OTL AlphaWRITE OUTLINE FORMAT ;
                          SCREEN
         OFFSET     1$    ; M$OT1 - hints
         OFFSET     2$    ; M$NUM Arabic Numerals
         OFFSET     3$    ; M$UCA Upper-Case Alphabetics
```

Any text following a semicolon is termed a "comment" and is ignored by the Assembler program.

One `OFFSET` line must be defined for each message. The order of the `OFFSET` lines is significant and must not be changed. The `OFFSET` statement takes a label as its only parameter. That label must be unique in the file, and must appear in the Message Table Area to mark the start of the corresponding message.

The Message Table Area

The Message Table Area follows the Message Dispatch Area. This area must be changed during editing to translate the file.

Each message defined in the Message Table Area starts at a label defined in the Message Dispatch Area. The message continues until a `BYTE 0` command or an `ASCIZ` command is found. The next message can start on the next line.

Although the lines in the Message Table Area may look (to an Assembler programmer) like they are defining machine code instructions (or macros) that will be executed at runtime, it is important to realize that each message is just a byte stream that will be interpreted by the AlphaWRITE Message Manager. Each seeming command, such as `CRT` and `POS,` are macros, defined in `MSGSYM.M68`, that expand to a series of `BYTE` statements.

When AlphaWRITE follows the chain of pointers down to the Message Table Area and a particular message, it starts interpreting each byte, outputting a character or carrying out a screen operation for each byte processed until it encounters a null byte. Message processing then terminates, and AlphaWRITE carries on with its next operation.

The various commands in the Message Area let you easily define whatever text, cursor positioning, and screen attribute commands you want for each message. AlphaWRITE will interpret your commands at execution time.

Here is an example of a Message Table area:

```
0$:     CRT       -1,0
        CRT       -1,12
        POS       1,26
        CRT       -1,32
        ASCII     " AlphaWRITE OUTLINE FORMAT "
        CRT       -1,33
        BYTE      0
1$:     CRT       -1,12
        POS       23,22
        ASCII     "[ Press EXECUTE to update outline ]"
        POS       24,15
        ASCIZ     "[ Press MENU or CANCEL to leave outline
                  unchanged ]"
2$:     ASCIZ     "Arabic Numerals"
        ...
        END
```

Each message begins with a *label*, which in this case is a number followed by a dollar sign and colon, e.g., `0$:`, `1$:`, etc. The label itself, without the colon, must correspond with the parameter of an `OFFSET` statement in the Message Dispatch Area for example, `0$`, `1$`, etc.

After the label are the statements that define the byte stream of the each message: `ASCII`, `ASCIZ`, etc. Each statement must be placed on a separate line. The following statements are legal and reasonable to include:

- `CRT`
- `POS`
- `BYTE`
- `ASCII`
- `ASCIZ`

The CRT Statement

`CRT` statements perform a special TCRT function, just as `PRINT TAB(-1,n)` statements function in AlphaBASIC. The keyword `CRT` takes two decimal numbers (without decimal points) as arguments, separated by a comma. For example:

```
CRT         -1,0
```

A list of the `CRT` codes can be found in the Terminal Service System section of the *AMOS Monitor Calls* manual.

The POS Statement

`POS` statements position the cursor, just as `PRINT TAB(row,col)` statements function in AlphaBASIC. The keyword `POS` takes two decimal numbers (without the decimal point) as arguments, separated by a comma.

```
POS         23,40
```

Mode vs. Field Terminals

Traditional terminals, like the AM-60 and AM-62A, are **field** terminals. Their handling of the underscore, reverse video, and blink calls are based on attributes that take up one character position on the screen display. Newer types of terminals, such as the AM-72 and ones accompanying PCs, are **mode** terminals. Mode terminals treat these functions just like low and high intensity: you turn a function on, display text, and turn it off. The attributes on a mode terminal do not take up a screen position.

AlphaWRITE's message system automatically outputs the proper display sequences and control codes for both mode and field terminals. However, this is founded upon the assumption that the message files are specifically coded in the manner described below.

Reverse video, underscore, and blink calls should be made as for a mode terminal, but with attributes occupying a space. For every attribute you turn on in a message, you must also turn off before AlphaWRITE finishes processing the message's byte stream. This must be done even if you know that the next message to be displayed will turn on the same attributes that the current message will use.

For example, consider a message that outputs the text "AlphaWRITE" to the screen, in reverse video, at position `(1,35)`. The message to perform this is:

```
135$:   POS         1,34
        CRT         -1,32
        ASCII       "AlphaWRITE"
        BYTE        -1,33
        BYTE        0
```

Let's consider each element of the message:

POS	1,34	Position the cursor at (1,34)
CRT	-1,32	Output a reverse video sequence, which occupies a space. This leaves the cursor at (1,35)
ASCII	"AlphaWRITE"	Display the text.
CRT	-1,33	Turn off reverse video.
BYTE	0	End of message.

The BYTE Statement

BYTE statements are used to define characters that make up a string or will appear on the screen. To define consecutive ASCII characters, it is quicker and easier to use the ASCII or ASCIZ statements defined below. However, these latter two statements may not allow you to define characters whose value lies above 126.—the range of many ISO Latin-1 characters.

The BYTE statement takes one or more parameters separated by commas. The parameters should be decimal numbers with a decimal point after the number that represents the decimal number of the ISO Latin-1 character you want to use. For example, to define o_circumflex, you use:

```
BYTE        244.
```

To define three consecutive o_circumflexes, you would use:

```
BYTE        224.
BYTE        224.
BYTE        224.
```

Or, you can define more than one character in the same statement. The characters do not have to have the same value to do this:

```
BYTE        244., 244., 244.
```

Each decimal number has a trailing decimal point, and if there is more than one number defined in the same statement, there must be a comma separating one number from the next. A comma must not appear at the end of the list, however.

A special value of zero is used to signal the end of the stream for a particular message:

```
BYTE        0
```

If you omit the BYTE 0 statement at the end of a message, AlphaWRITE will happily carry on processing the byte stream; executing the next message, and the next, until it encounters a BYTE 0 statement.

BYTE 0 can also be hidden inside the ASCIZ statement; see the ASCIZ section for further details.

The ASCII Statement

If you have a number of ASCII characters to define in a message, it is tedious to have to look up the decimal value of each character and use the `BYTE` statement. The `ASCII` statement allows you to use a shorthand method of specifying consecutive ASCII characters. Internally, the Assembler processes the `ASCII` statement as a `BYTE` statement for each character in the `ASCII` statement.

`ASCII` is followed by text enclosed in delimiters. A delimiter may be any printable character not contained in the text itself. Any character that your version of the Assembler and AlphaVUE (or AlphaXED) will accept can be entered. In the following example, the first statement's delimiters are quotation marks; the second's are backslashes.

```
ASCII "Enter printer number:"
ASCII /Too Long/
```

The result of these two consecutive statements is to define the string "Enter printer number:Too long".

If you want to enter a printable character in the range 160.-255. (an ISO Latin-1 character), you may have to use a combination of `ASCII` and `BYTE` commands. For example, to cause the word "rôle" to be output, you may have to use:

```
ASCII      /r/
BYTE       244.
ASCII      /le/
```

where `BYTE 244.` (note the decimal point) is the decimal value corresponding to the ISO Latin-1 character "ô". (See Appendix B for a list of the decimal values corresponding to ISO Latin-1 characters.) Because AlphaWRITE interprets the message as a stream of bytes, this splitting of a word into a number of pieces is perfectly legal.

The ASCIZ Statement

`ASCIZ` statements are like `ASCII` in that they define text but `ASCIZ` statements imply an automatic `BYTE 0` at the end of the text they are defining.

You can use `ASCIZ` as a convenient method of ending a message. Because it automatically contains the BYTE 0 statement, `ASCIZ` may only be used as the final component of a message. As an example, instead of writing:

```
0$:   POS        22,1
      ASCII      "Enter command name and press RETURN:"
      BYTE       0
1$:   ....
```

You can write:

```
0$:   POS        22,1
      ASCIZ      "Enter command name and press RETURN:"
1$:   ....
```

However, the following use of `ASCIZ` is incorrect. Because of the automatic `BYTE 0` at `ASCIZ`'s end, only the words "I'm displayed" will be output when the message is displayed. The words " but I'm missing" and the `CRT -1,33` command will never appear. Note the `BYTE 0` statement which will terminate the message that starts at `0$:`. You cannot use an `ASCIZ` statement to terminate the message stream in this example because the previous statement does not display text, where you would use the `ASCII` command, it turns off reverse video.

```
0$:   POS        20,1
      CRT        -1,32
      ASCIZ      "I'm displayed"
      ASCII      " but I'm missing"
      CRT        -1,33
      BYTE       0
1$:
      ....
```

Translating The Message Area

The areas that need adjusting when translating the file should now be clear.

- The `ASCII`, `ASCIZ`, and `BYTE` statements should be translated first.
- Then `POS` statements may need adjusting if the length of the original text is not the same as the newly translated text.

Example One

As an example, let's translate the following fragment from English into German:

```
2$:     POS        1,31       ; center the message....
        ASCIZ      /Delete a file/
3$:     ....
```

Note that as the last statement is an `ASCIZ` statement, the `BYTE 0` is automatically included as the last part of the message stream. The example is equivalent to:

```
2$:     POS        1,31       ; center the message....
        ASCII      /Delete a file/
        BYTE       0
3$:     ....
```

The German for "Delete a file" is "Datei löschen". It happens to have the same number of characters in it as the English version. Therefore the `POS` statement will not have to be adjusted.

We also need to deal with the o_umlaut character ("ö") by splitting the text into an `ASCII` statement for characters up to the o_umlaut, using a `BYTE` statement for the o_umlaut. The decimal value of "ö" is 246. and then an `ASCIZ` statement for the final part of the text. This gives us:

```
2$:     POS        1,31       ; center the message....
        ASCII      /Datei l/
        BYTE       246.
        ASCIZ      /schen/
3$:     ....
```

We would also have changed the `OBJNAM` directive at the top of the file to reflect the correct file extension for a German language file. The file extension to use would be defined in the AMOS `.LDF` language definition file for the German language.

Example Two

As a second example, we will translate the following example into French:

```
10$:    POS        1,31       ; center the message....
        ASCIZ      /The file is locked/
11$:    ....
```

The French for "The file is locked" is "Le fichier est verrouillé". This string has 24 characters in it, compared to 18 in the English version. It also has an non-ASCII character, but one in the ISO Latin-1 Alphabet, the e_acute character, "é". The decimal value of e_acute is 233.

We need to adjust the `POS` statement to center the text properly, and we'll use a `ASCII, BYTE` sequence to deal with the accented character. We also have to be careful to ensure that a `BYTE 0` is in place at the end of the message stream when we're done. At present, it is in the `ASCIZ` statement, hidden at the end of the text. But now we have to put the last character of the translation (the e_acute) into a `BYTE` statement, so we have to include another `BYTE 0` statement explicitly in the message stream—it can no longer "hide" in an `ASCIZ` statement.

Here is the final version:

```
10$:    POS        1,25       ; center the message....
        ASCII      /Le fichier est verrouill/
        BYTE       233., 0    ; no comma at end
11$:    ....
```

Note that we've defined two characters with one `BYTE` statement. We could have written this example as:

```
10$:    POS        1,25       ; center the message....
        ASCII      /Le fichier est verrouill/
        BYTE       233.       ; no comma
        BYTE       0
11$:    ....
```

The two are equivalent. Again, we would need to change the OBJNAM Assembler directive to the correct extension for a French language file.

Example Three

As a final example, let's add some screen attribute statements to the above example:

```
10$:    POS        1,30       ; center the message....
        CRT        -1,32      ; ....assume attribute space
                              ; for reverse video
        ASCII      /The file is locked/
        CRT        -1,33      ; turn off reverse video
        BYTE       0
3$:     ....
```

Note that the POS command has been offset one character to allow for the attribute space on the screen. The ASCIZ statement has had to be turned into an ASCII statement, because it is not longer the last statement in the message stream—the CRT -1,33 statement holds that position. Therefore an explicit BYTE 0 statement is required to terminate the message stream.

Now we can translate the message into French as follows:

```
10$:    POS        1,24       ; center the message....
        CRT        -1,32      ; ....assume attribute space
                              ; for reverse video
        ASCII      /Le fichier est verrouill/
        BYTE       233.
        CRT        -1,33      ; turn off reverse video
        BYTE       0
3$:     ....
```

Note that the BYTE 233. statement cannot have the "0" added, as we need to execute the CRT -1,33 statement in the message stream.

COMMAND AREA

The Command Area consists of two parts:

- The Command Dispatch Area.
- The Command Table Area.

The Command Dispatch Area must appear after the Dispatch Area, and the Command Table Area must appear after the Command Dispatch Area. Conventionally, both sections appear after the Message Area.

Only the Command Table Area needs to be changed when translating the file to a different language. The Command Dispatch Area remains the same.

Command Dispatch Area

The format of the Command Dispatch Area is identical to that of the Message Dispatch Area. It too uses the `OFFINI` and `OFFSET` commands. The label preceding the `OFFINI` command must be the same label as used in the `OFFSET` command in the Dispatch Area that pointed to the Command Dispatch Area. The labels on the `OFFSET` statements must be unique in the file, and must point to the start of the corresponding command in the Command Table Area. As an example:

```
CMD:    OFFINI
        OFFSET     100$            ; point to first command
        OFFSET     101$            ; point at second command
        OFFSET     102$            ; point at third command
```

The label—the word before the colon in the `OFFINI` line—uses the same word as in the Dispatch Area's `OFFSET CMD` line. Any text following a semicolon is termed a "comment" and is ignored by the Assembler program.

One `OFFSET` line must be defined for each command. The order of the `OFFSET` lines is significant and must not be changed. The `OFFSET` command takes a label as its only parameter. That label must be unique in the file, and must appear in the Command Table Area to mark the start of the corresponding command.

Command Table Area

The Command Table Area follows the Command Dispatch Area. This area must be changed during editing to translate the file.

The format of the Command Table Area is different to that of the Message Table Area. AlphaWRITE treated the contents of the Command Message Table area to define strings to be output or as lists of valid responses. Therefore, you cannot put in `CRT` screen attribute statements or `POS` cursor positioning statements—they have no place here.

However, `ASCII`, `ASCIZ` and `BYTE` statements can be used there to define the strings. You must take note of the existing structure of the Command Table Area. In many files, a label (such as `0$:`) starts the definition of a number of strings, each string being an `ASCIZ` string. The next label (such as `1$:`) may be a number of `ASCIZ` statements away. Make sure you preserve that structure as you translate the file. All the instructions, examples and warnings covering the `ASCII`, `ASCIZ` and `BYTE` statements given in the Message Table Area section apply here.

Translating the Command Table Area

The translation process for the Command Table Area is very similar to that for the Message Table Area. It is simpler because there are no cursor positioning or screen attribute statements to worry about.

Each `ASCII`, `ASCIZ`, and `BYTE` statement needs to be translated. If you need to insert a `BYTE` statement to insert an ISO Latin-1 character, you must make sure that you do

not disturb the location of the `BYTE 0` statement, whether it is explicitly used or hidden in an `ASCIZ` statement.

FIELD AREA

The Field Area consists of two parts:

- The Field Dispatch Area
- The Field Table Area

The Field Dispatch Area must appear after the Dispatch Area, and the Field Table Area must appear after the Message Dispatch Area. Conventionally, both sections appear after the Command Area.

Only the Field Table Area needs to be changed when translating the file to a different language. The Field Dispatch Area remains the same.

Field Dispatch Area

The format of the Field Dispatch Area is identical to that of the Message Dispatch Area. It too uses the `OFFINI` and `OFFSET` commands. The label preceding the `OFFINI` command must be the same label as used in the `OFFSET` command in the Dispatch Area that pointed to the Field Area. The labels on the `OFFSET` statements must be unique in the file, and will point to the start of the corresponding command in the Command Table Area. As an example:

```
FLD:    OFFINI
        OFFSET     200$       ; point to first field info
        OFFSET     201$       ; point to second field info
        OFFSET     202$       ; point to third field info
```

The label—the word before the colon in the `OFFINI` line—uses the same word as in the Dispatch Area's `OFFSET FLD` line. Any text following a semicolon is termed a "comment" and is ignored by the Assembler program.

One `OFFSET` line must be defined for each message. The order of the `OFFSET` lines is significant and must not be changed. The `OFFSET` command takes a label as its only parameter. That label must be unique in the file, and must appear in the Field Table Area to mark the start of the corresponding field.

Field Table Area

The Field Table Area follows the Field Dispatch Area. This area must be changed during editing to translate the file.

The Field Table Area is used to control user data entry. Each statement in this area is a `FIELD` statement, which takes six parameters. An example of this area is shown below:

```
;                R  C  M  M  T     F
;                O  O  I  A  Y     L
;                W  L  N  X  P     G

200$:   FIELD    23,40,01,36,F$STR,F$HLP!F$UCS    ; outer

201$:   FIELD    03,01,01,40,F$STR,0              ; inner

202$:   FIELD    01,35,01,20,F$STR,0              ; version
```

Any text on a line after a semicolon is treated as a comment and ignored by the Assembler. Each line starts with a label which must correspond with the parameter of an `OFFSET` command in the Field Dispatch Area. All numbers in the `FIELD` statement are decimal numbers, and appear without a decimal point.

The first two parameters are the cursor row and column position for the user's input.

The third and fourth parameter define the minimum and maximum number, respectively, of characters the user can enter into the field.

The fifth and sixth parameters define the types of data that can be entered, and the attributes of that data. These two parameters must not be altered.

Translating the Field Dispatch Area

To translate the Field Dispatch Area, you must know how many characters the user needs to enter in the desired lanaguage to satisfy the prompt. You need to alter the third and fourth parameters accordingly.

AlphaWRITE has to read the user's input into its own internal variables. These variables have a finite length. If you allow too many characters to be input into the field, the user's input will overwrite other data areas in AlphaWRITE and cause problems. Do not increase the maximum field lengths too much!

FINAL ASSEMBLER DIRECTIVES

This area does not require translation.

The last two statements in the file should be:

```
EVEN
END
```

The `EVEN` statement is a command to the Assembler to carry out an internal operation. The `END` statement tells the Assembler to stop reading any further in the source `.M68` file and move on to the next phase of its operations.

Both statements are required at the end of the file.

Make sure that you do not include any statements after the `END` statement—the Assembler will ignore them!

ASSEMBLING A MESSAGE FILE

You must have the version 2.x Assembler system distributed with AMOS versions 1.4 and later, and versions 2.0 and later in order to assemble message files. Message files include a file named `MSGSYM.M68`, which must be present in order for the assembly to be successful. This file is usually held in DSK0:[7,7] (MAC:).

As mentioned earlier, the assembled message file has the same name as the source `.M68` file. The extension of the assembled file is controlled by the `OBJNAM` directive at the start of the message file. The argument should be changed from `USA` to the standard extension for the target language as defined in the target language's language definition file (`.LDF`).

```
OBJNAM      .UK
```

To assemble the `.M68` source file, which creates the language-specific message file, use the `M68` command. For example:

```
M68 WRT400 RETURN
```

HELP FILES

The AlphaWRITE software comes with a series of `.USA` files which you can directly modify, or use as a guideline for translation to a another language. These files are free-format files that can be directly edited with AlphaVUE, or AlphaXED.

The table below provides the help file name and its description. Footnotes are located at the end of the table.

FILENAME	DESCRIPTION
BLDPRF.HLP	BLDPRF Utility System Level Help
DICSRT.HLP	DICSRT Utility System Level Help
WRITE.HLP	WRITE Command System Level Help
WRTCLP.HLP	Command line processing System Level Help
WRTDIR.HLP	WRTDIR Command System Level Help
WRTINI.HLP	Initialization file System Level Help
WHM001.USA	Main Menu Help Topic 1
WHM002.USA	Main Menu Help Topic 2
WHM003.USA	Main Menu Help Topic 3
WHM004.USA	Main Menu Help Topic 4
WHM005.USA	Main Menu Help Topic 5
WHM006.USA	Main Menu Help Topic 6
WHM007.USA	Main Menu Help Topic 7
WHM008.USA	Main Menu Help Topic 8
WHM009.USA	Main Menu Help Topic 9
WHM010.USA	Main Menu Help Topic 10
WHM011.USA	Main Menu Help Topic 11
WHM012.USA	Main Menu Help Topic 12

FILENAME	DESCRIPTION
WHM013.USA[1]	Main Menu Help Topic 13
WHM014.USA[2]	Main Menu Help Topic 14
WHM015.USA[3]	Main Menu Help Topic 15
WHE001.USA	Editing Menu Help Topic 1
WHE002.USA	Editing Menu Help Topic 2
WHE003.USA	Editing Menu Help Topic 3
WHE004.USA	Editing Menu Help Topic 4
WHE005.USA	Editing Menu Help Topic 5
WHE006.USA	Editing Menu Help Topic 6
WHE007.USA	Editing Menu Help Topic 7
WHE008.USA	Editing Menu Help Topic 8
WHE009.USA	Editing Menu Help Topic 9
WHE010.USA	Editing Menu Help Topic 10
WHE011.USA	Editing Menu Help Topic 11
WHE012.USA	Editing Menu Help Topic 12
WHE013.USA	Editing Menu Help Topic 13
WHE014.USA	Editing Menu Help Topic 14
WHE015.USA	Editing Menu Help Topic 15
WHE016.USA	Editing Menu Help Topic 16
WHE017.USA	Editing Menu Help Topic 17
WHE018.USA	Editing Menu Help Topic 18
WHE019.USA	Editing Menu Help Topic 19
WHE020.USA	Editing Menu Help Topic 20
WHE021.USA	Editing Menu Help Topic 21
WHE022.USA	Editing Menu Help Topic 22
WHE023.USA	Editing Menu Help Topic 23
WHE024.USA	Editing Menu Help Topic 24
WHM013.USA[1]	Editing Menu Help Topic 25
WHM014.USA[2]	Editing Menu Help Topic 26
WHM015.USA[3]	Editing Menu Help Topic 27
WHT001.USA	Text Display Help Topic 1
WHT002.USA	Text Display Help Topic 2
WHT003.USA	Text Display Help Topic 3
WHT004.USA	Text Display Help Topic 4
WHT005.USA	Text Display Help Topic 5
WHT006.USA	Text Display Help Topic 6
WHT007.USA	Text Display Help Topic 7
WHT008.USA	Text Display Help Topic 8
WHT009.USA	Text Display Help Topic 9
WHT010.USA	Text Display Help Topic 10
WHT011.USA	Text Display Help Topic 11
WHT012.USA	Text Display Help Topic 12
WHT013.USA	Text Display Help Topic 13
WHT014.USA	Text Display Help Topic 14
WHT015.USA	Text Display Help Topic 15
WHT016.USA	Text Display Help Topic 16
WHT017.USA	Text Display Help Topic 17
WHT018.USA	Text Display Help Topic 18
WHT019.USA	Text Display Help Topic 19
WHT020.USA	Text Display Help Topic 20
WHT021.USA	Text Display Help Topic 21
WHT022.USA	Text Display Help Topic 22
WHM013.USA[1]	Text Display Help Topic 23

FILENAME	DESCRIPTION
WHM014.USA[2]	Text Display Help Topic 24
WHM015.USA[3]	Text Display Help Topic 25
WHH001.USA	Doc Hdr Mini Help Description
WHH002.USA	Doc Hdr Mini Help Author
WHH003.USA	Doc Hdr Mini Help Operator
WHH004.USA	Doc Hdr Mini Help Department
WHH005.USA	Doc Hdr Mini Help Protection
WHH006.USA	Doc Hdr Mini Help Password
WHH007.USA	Doc Hdr Mini Help Leading
WHH008.USA	Doc Hdr Mini Help Page Length
WHH009.USA	Doc Hdr Mini Help Top Margin
WHH010.USA	Doc Hdr Mini Help Title Length
WHH011.USA	Doc Hdr Mini Help Text Length
WHH012.USA	Doc Hdr Mini Help Spc Bef. Footnotes
WHH013.USA	Doc Hdr Mini Help Footer Length
WHH014.USA	Doc Hdr Mini Help Printer Form Name
WHH015.USA	Doc Hdr Mini Help Formatted
WHH016.USA	Doc Hdr Mini Help Left Paper Offset
WHH017.USA	Doc Hdr Mini Help Right Justify
WHH018.USA	Doc Hdr Mini Help Character Spacing
WHH019.USA	Doc Hdr Mini Help Word Spacing
WHH020.USA	Doc Hdr Mini Help Line Spacing
WHH021.USA	Doc Hdr Mini Help Footnote Print Type
WHH022.USA	Doc Hdr Mini Help Footnote Separator
WHH023.USA	Doc Hdr Mini Help Kerning
WHH024.USA	Doc Hdr Mini Help Orientation
WHS001.USA	Mode Cmd Mini Help Fold Case
WHS002.USA	Mode Cmd Mini Help Menu Bypass
WHS003.USA	Mode Cmd Mini Help Character Insert
WHS004.USA	Mode Cmd Mini Help Line Insert
WHS005.USA	Mode Cmd Mini Help Quick Dct.
WHS006.USA	Mode Cmd Mini Help Standard Dct.
WHS007.USA	Mode Cmd Mini Help Professional Dct.
WHS008.USA	Mode Cmd Mini Help Personal Dct.
WHS009.USA	Mode Cmd Mini Help Reformat
WHS010.USA	Mode Cmd Mini Help Font Sub.
WHS011.USA	Mode Cmd Mini Help On-Screen Att.
WHS012.USA	Mode Cmd Mini Help Color
WHS013.USA	Mode Cmd Mini Help High/Low
WHS014.USA	Mode Cmd Mini Help Prop. fonts
WHS015.USA	Mode Cmd Mini Help Journaling
WHS016.USA	Mode Cmd Mini Help Keystrokes
WHS017.USA	Mode Cmd Mini Help Maximum keystrokes
WHS018.USA	Mode Cmd Mini Help Verb Dictionary

FOOTNOTES

1. Main Menu Help Topic 13 is the same as Editing Menu Topic 25 and Text Display Topic 23; WHM013.USA is displayed by each.
2. Main Menu Help Topic 14 is the same as Editing Menu Topic 26 and Text Display Topic 24; WHM014.USA is displayed by each.
3. Main Menu Help Topic 15 is the same as Editing Menu Topic 27 and Text Display Topic 25; WHM015.USA is displayed by each.

CHAPTER FIVE
EXTENDED CHARACTERS

DEFINING AN EXTENDED CHARACTER SET

In order to support non-standard characters, AlphaWRITE allows the use of an extended character set. In addition to the printable ISO Latin 1 character set (decimal 32 through decimal 126 inclusive), up to 999 characters may be defined and used. The definition of extended characters generally consists of two steps:

1. Create an extended character definition table.

2. Modify the function key translation file (.WRX).

CREATING AN EXTENDED CHARACTER DEFINITION TABLE

WRT400.*lng* is the main message file for AlphaWRITE (where *lng* represents the current language.). The source to this file, WRT400.M68, is provided with the software release. In order to define extended characters, it is necessary to locate and modify the extended character table in WRT400.M68.

You use AlphaVUE or AlphaXED to work with the WRT400.M68 file. You can find the table by using searching for the "extended character display table." The extended character definition table uses the format:

```
WORD character_code

     BYTE       b1,b2,b3,b4,b5,b6,b7,b8
```

The above example defines an extended character code and the bytes used to display it. The eight-byte sequence follows standard message file rules, which means it can contain characters and/or TCRT calls. The size of this field is fixed; null bytes must be defined, if necessary, to insure a size of eight bytes.

A zero word terminates the table. For example:

```
;EXTENDED CHARACTER TABLE
WORD     100.                        ; code for A_accent
BYTE     -1,175.,0,0,0,0,0,0         ; display codes
WORD     101.                        ; code for E_accent
BYTE     'E,0,0,0,0,0,0,0            ; display codes
WORD     102.                        ; code for O_caret
BYTE     -1,23.,-1,64.,-1,24.,0,0    ; display codes
WORD     0                           ; ** end of table **
```

The version of WRT400.M68 released from Alpha Micro does not contain a table.

MAPPING EXTENDED CHARACTERS TO THE KEYBOARD

In conjunction with the extended character table (described above) AlphaWRITE provides an easily set-up system for mapping keyboard keys to extended characters.

The universal sequence (ESC # nnn RETURN) allows no-echo entry of an extended character sequence at AlphaWRITE's document text display. All that is required to make extended characters usable, is then to map each key to a sequence of this format.

The general procedure for mapping a key is to use FIXTRN on a function key translation file (.WRX). FIXTRN is an AMOS utility which is documented in the *AMOS Commands Reference Manual*.

Generally, after invoking FIXTRN:

1. Press the key you want to use as the signal to end you input.
2. Press they key that you want to assign to the extended character.
3. Press ESC, press # (pound sign), the character sequence (a number from 0 to 999), and press RETURN.
4. Press the terminating character you chose in step 1.
5. Proceed to the next key, repeating steps 2 through 5 for all the keys you want to map.

An Extended Character Example

A German user has a terminal capable of generating and displaying an umlaut. There are keys for A umlaut, O umlaut, and U umlaut on the terminal.

After defining a language definition file for German, and translating AlphaWRITE's message files into German, the dealer modifies the extended character table in the WRT400 file. Arbitrarily, the following extended character codes are chosen:

1	A_umlaut (upper case)
2	O_umlaut (upper case)
3	U_umlaut (upper case)
4	A_umlaut (lower case)
5	O_umlaut (lower case)
6	U_umlaut (lower case)

The WRT400 extended character table is modified so that the codes necessary to display these characters on the customized terminal are defined.

The FIXTRN program is invoked on the function key translation file for the customized terminal. The special umlaut keys are mapped as follows:

A_umlaut (upper case) → ESC # 1 CR
O_umlaut (upper case) → ESC # 2 CR
U_umlaut (upper case) → ESC # 3 CR
A_umlaut (lower case) → ESC # 4 CR
O_umlaut (lower case) → ESC # 5 CR
U_umlaut (lower case) → ESC # 6 CR

SCOPE OF EXTENDED CHARACTERS

Extended characters provide a mechanism for teaching AlphaWRITE about new characters to be entered from the keyboard, displayed on the screen, and printed. AlphaWRITE's understanding of extended characters does not provide universal support for these characters. For example, the **SORT** command does not take extended characters into account as language characters; the **SPELL** command does not recognize extended characters as part of a word.

For support of foreign languages, using the ISO Latin 1 character set is preferable to using extended characters. All of AlphaWRITE 2.1's (and later) features recognize and support characters belonging to the ISO Latin 1 character set.

CHAPTER SIX
THE AWREAD XCALL

The AWREAD.SBR XCALL subroutine, which accompanies the AlphaWRITE software, reads an AlphaWRITE file, either one line at a time or all lines at once.

The syntax for using this XCALL is:

```
XCALL AWREAD,RECORD'BUFFER,INPUT'CHANNEL,MODE'FLAG
```

WHERE	MEANS
RECORD'BUFFER	Buffer used to hold each record (line) of the AlphaWRITE file. Must be a string or unformatted variable large enough to receive the requested text.
INPUT'CHANNEL	The AlphaBASIC file channel number on which the document file has been opened. An Alpha Micro 6-byte floating point variable.
MODE'FLAG	This flag informs AWREAD which mode it is to use to read the AlphaWRITE file. An Alpha Micro 6-byte floating point variable whose valid values are: 0 All lines at once (read the rest of the document all at once). 1 One line at a time and include embedded text and other information blocks. 2 Bypass the document header and position the "read pointer" at the first text character. 3 Read next line, ignoring any embedded text or other information blocks. Note: A line is terminated by a NULL.

The document file must be opened by the AlphaBASIC program before using AWREAD, and closed after the last call to the subroutine.

CHAPTER SEVEN
INTERNATIONAL LANGUAGE SUPPORT: NRCS TO ISO LATIN-1 TRANSFORMATION

Traditionally, AlphaWRITE users who are not speakers of American English have used the **N**ational **R**eplacement **C**haracter **S**et (NRCS) technique. While NRCS has made the important foreign language characters available to users, this has been at the cost of "giving up" characters from the ASCII set (such as [or {).

Now that AlphaWRITE supports the ISO Latin-1 character set, this trade-off is no longer necessary. AlphaWRITE 2.1 and later, support ISO Latin-1 directly.

For the benefit of AlphaWRITE users who will be upgrading to AlphaWRITE 2.1 and later, a conversion facility is required to transform NRCS documents to ISO Latin-1. A sample transformation table is provided with AlphaWRITE 2.1 and later to illustrate the table's format. However, you must modify the transformation table for your language before converting documents. The transformation table is described below.

DOCUMENT CONVERSION

AlphaWRITE version 1.2A or version 2.0 documents are converted into AlphaWRITE 2.1 (and later) format when you use one of two methods:

1. AWCONV command to convert one or more AlphaWRITE 1.2A or 2.0 documents to 2.1 and later format.

2. The Main Menu **EDIT** command to convert an AlphaWRITE 1.2A or 2.0 document to 2.1 and later format.

In both cases, NRCS-to-ISO Latin-1 transformation takes place transparently and automatically *if* AlphaWRITE 2.1 or later locates a file in WRT: named NR2ISO.*lng* (where *lng* is the message file extension defined in your Language Definition File). If NR2ISO exists, it is used as a transformation table. If it does not exist, no character transformation takes place. American English, for example, requires no character transformation.

Documents older than AlphaWRITE 1.2A must first be converted to AlphaWRITE 1.2A before conversion to AlphaWRITE 2.0 and later can take place. If you have documents

made by AlphaWRITE version 1.2 or earlier, contact Technical Support for assistance in upgrading.

TRANSFORMATION TABLE FORMAT

The transformation table contains 128 single-byte codes, representing ASCII positions 0 through 127 inclusive. Each byte code is the replacement code that is used.

If a transformation table was being set up for American English (which is not necessary), with the intent of leaving all codes unchanged, the result would be a file containing bytes 0, 1, 2, 3, ..., 126, 127.

The technique used during document conversion is as follows: if a printable text character is encountered, the character code is added to the base of the table, and the resultant indexed byte is used as a replacement. This makes it easy to convert any character conventions currently in place (NRCS or otherwise).

SAMPLE TRANSFORMATION TABLE

The following is the sample transformation table source, NR2ISO.M68, which comes with AlphaWRITE 2.1, and later

```
;*****************************************************
;*
;*      NRCS to ISO_Latin_I Transformation Table
;*      American English
;*
;*****************************************************

OBJNAM         .USA

RADIX          10

BYTE    0              ; 0
BYTE    1              ; 1
BYTE    2              ; 2
BYTE    3              ; 3
BYTE    4              ; 4
BYTE    5              ; 5
BYTE    6              ; 6
BYTE    7              ; 7
BYTE    8              ; 8
BYTE    9              ; 9
BYTE    10             ; 10
BYTE    11             ; 11
BYTE    12             ; 12
BYTE    13             ; 13
BYTE    14             ; 14
BYTE    15             ; 15
BYTE    16             ; 16
BYTE    17             ; 17
BYTE    18             ; 18
BYTE    19             ; 19
BYTE    20             ; 20
BYTE    21             ; 21
BYTE    22             ; 22
BYTE    23             ; 23
BYTE    24             ; 24
BYTE    25             ; 25
BYTE    26             ; 26
BYTE    27             ; 27
BYTE    28             ; 28
BYTE    29             ; 29
BYTE    30             ; 30
BYTE    31             ; 31
```

```
BYTE    32              ; 32     SPACE
BYTE    33              ; 33     exclamation
BYTE    34              ; 34     double quote
BYTE    35              ; 35     pound sign
BYTE    36              ; 36     dollar sign
BYTE    37              ; 37     percent sign
BYTE    38              ; 38     ampersand
BYTE    39              ; 39     single quote
BYTE    40              ; 40     open parenthesis
BYTE    41              ; 41     close parenthesis
BYTE    42              ; 42     asterisk
BYTE    43              ; 43     plus
BYTE    44              ; 44     comma
BYTE    45              ; 45     minus
BYTE    46              ; 46     period
BYTE    47              ; 47     slash
BYTE    48              ; 48     0
BYTE    49              ; 49     1
BYTE    50              ; 50     2
BYTE    51              ; 51     3
BYTE    52              ; 52     4
BYTE    53              ; 53     5
BYTE    54              ; 54     6
BYTE    55              ; 55     7
BYTE    56              ; 56     8
BYTE    57              ; 57     9
BYTE    58              ; 58     colon
BYTE    59              ; 59     semicolon
BYTE    60              ; 60     left angle bracket
BYTE    61              ; 61     equal
BYTE    62              ; 62     right angle bracket
BYTE    63              ; 63     question mark
BYTE    64              ; 64     at sign
BYTE    65              ; 65     A
BYTE    66              ; 66     B
BYTE    67              ; 67     C
BYTE    68              ; 68     D
BYTE    69              ; 69     E
BYTE    70              ; 70     F
BYTE    71              ; 71     G
BYTE    72              ; 72     H
BYTE    73              ; 73     I
BYTE    74              ; 74     J
BYTE    75              ; 75     K
BYTE    76              ; 76     L
BYTE    77              ; 77     M
BYTE    78              ; 78     N
BYTE    79              ; 79     O
BYTE    80              ; 80     P
BYTE    81              ; 81     Q
BYTE    82              ; 82     R
BYTE    83              ; 83     S
BYTE    84              ; 84     T
BYTE    85              ; 85     U
BYTE    86              ; 86     V
BYTE    87              ; 87     W
BYTE    88              ; 88     X
BYTE    89              ; 89     Y
BYTE    90              ; 90     Z
BYTE    91              ; 91     left square bracket
BYTE    92              ; 92     backslash
BYTE    93              ; 93     right square bracket
BYTE    94              ; 94     caret
BYTE    95              ; 95     underscore
BYTE    96              ; 96     grave accent
BYTE    97              ; 97     a
BYTE    98              ; 98     b
BYTE    99              ; 99     c
BYTE    100             ; 100    d
BYTE    101             ; 101    e
BYTE    102             ; 102    f
BYTE    103             ; 103    g
BYTE    104             ; 104    h
BYTE    105             ; 105    i
BYTE    106             ; 106    j
BYTE    107             ; 107    k
```

```
BYTE    108             ; 108    l
BYTE    109             ; 109    m
BYTE    110             ; 110    n
BYTE    111             ; 111    o
BYTE    112             ; 113    p
BYTE    113             ; 112    q
BYTE    114             ; 114    r
BYTE    115             ; 115    s
BYTE    116             ; 116    t
BYTE    117             ; 117    u
BYTE    118             ; 118    v
BYTE    119             ; 119    w
BYTE    120             ; 120    x
BYTE    121             ; 121    y
BYTE    122             ; 122    z
BYTE    123             ; 123    left curly brace
BYTE    124             ; 124    vertical bar
BYTE    125             ; 125    right curly brace
BYTE    126             ; 126    tilde
BYTE    127             ; 127    del
EVEN

END
```

CHAPTER EIGHT
CHARACTER COMPOSITION AND ISO LATIN-1 SUPPORT

ENTERING ISO CHARACTERS VIA THE TERMINAL

AlphaWRITE 2.1 and later support 8-bit characters—specifically, the ISO Latin-1 character set. The AM-65 terminal supports this character set when it is used in native mode.

On the AM-65, the additional printable characters (A-accent, yen symbol, etc.) are created by *composing*. On an AM-65 terminal, to compose a character, you press [CTRL]-[SHIFT]-[FUNC] **and** then a two-letter sequence. For example, to enter an O-umlaut, the sequence is

[CTRL]-[SHIFT]-[FUNC] [O] ["]

Generating ISO Latin-1 characters requires an operating system version supporting 8-bit characters: AMOS 1.4, AMOS 2.2, or later.

ENTERING ISO CHARACTERS VIA AlphaWRITE SOFTWARE

An escape sequence, introduced with AlphaWRITE 2.1, makes ISO Latin-1 characters available to users by way of character composition. This feature allows characters to be composed, but under the control of AlphaWRITE.

AlphaWRITE's character composition is terminal and operating system independent because it can be performed on any terminal regardless of the current operating system version.

AlphaWRITE's compose-character escape sequence is

[ESC] [´] xx

where xx represents the same two letter codes used by the AM-65 terminal. For example, you use these keys to enter O-umlaut:

[ESC] [´] [O] ["]

AlphaWRITE's on-line Main, Editing, and Text display help menu topic "Composing International Characters" contains a complete list of two-character codes and the character they compose.

The *AlphaWRITE Reference Guide*'s Chapter 7 - "Printing Features" describes using the compose-character feature and Appendix P - "ISO Latin-1 Characters and Laser Printing" describes character availability.

HOW CHARACTERS ARE DISPLAYED

Regardless of whether you enter an ISO character using the terminal's composition sequence (hardware) or AlphaWRITE's compose-character feature (software), the display rules are the same:

1. If you are using an 8-bit terminal, you see the character requested.

2. If you are using a 7-bit terminal, a table is used to present alternative characters that are part of ASCII. For example, a-grave, a-accent, and a-umlaut all become just the letter "a", because that is the closest character available. When the cursor is positioned on the character, is to see a corresponding message on the status line. The message identifies this particular character by name, "a-grave," "ae ligature," and so on.

When combined with the **N**ational **R**eplacement **C**haracter **S**et (NRCS) setup options in the AM-62A, a good job can be done to map a single language's characters from the 8-bit ISO Latin-1 alphabet to the terminal's 7-bit NRCS equivalents.

Remember, an AM-65 is a 7-bit device when used in AM-62A emulation mode.

HOW CHARACTERS ARE PRINTED

Terminal support of 8-bit characters is not related to their printing. When you print a document, the printer driver supplies all of its available characters. Impact printers generally have limited capabilities in this area, but laser printers supply a high degree of compatibility.

PostScript Laser Printing

PostScript printers have their own character set which resembles, but is not identical to, ISO Latin-1. The AlphaWRITE 2.1 (and later) version of the LASWRT printer driver re-encodes PostScript's character set to a facsimile of ISO Latin-1. The end result directly supports all but a few of the ISO Latin-1 printable characters.

HP Laser Printing

HP LaserJet printers support a character set known as Roman-8. This is also similar to ISO Latin-1, but not identical. A translation process in the AlphaWRITE 2.1 (and later) version of the printer driver transparently maps ISO Latin-1 to Roman-8. Like the PostScript driver, there are a few characters that are unavailable.

Substitution

Many logical replacements are available when a certain character is not available on a certain printer. For example, the ISO Latin-1 "multiplication symbol" character is not available on HP or PostScript printers, but substituting a lower case "x" produces acceptable output.

Printers Not Supported

AlphaWRITE 2.1 and later does not provide ISO Latin-1 support in the MISC, AM301, AM302, or AM306x printer drivers. The ASCII subset that has always been supported will still print, and logical substitutions will be made for the ISO Latin-1 extensions.

LANGUAGE DEFINITION FILES

Language definition files must be defined as described in the *AMOS System Operator's Guide* so AlphaWRITE's features can fully support international characters during **SPELL**, **SEARCH**, **REPLACE**, **GLOBAL**, **SORT**, REVERSE CASE,

CHARACTER SET TRANSLATION TABLES

When AlphaWRITE 2.1 and later is invoked, it searches for a translation table for character set conversion *if* it determines that your terminal is a 7-bit device. If your terminal is an 8-bit device, no character set translation is required.

The name of the translation table is {driver}.{*lng*}, where *lng* is the message file extension defined in your Language Definition File. Examples of valid translation tables are AM62A.USA; AM60.UK; and AM70.F. These tables reside in the WRT: account.

If the terminal is a 7-bit device and AlphaWRITE cannot locate the proper translation table, it loads TERM.*lng* by default. In American English, this is TERM.USA.

HOW AlphaWRITE DETERMINES 7-BIT VS. 8-BIT TERMINALS

Under AMOS 1.4, 2.2, and later, there are flags indicating whether or not a terminal is an 8-bit device. If the terminal is a 7-bit device, AlphaWRITE attempts to load first {tdv}.{*lng*} and then (if that file is not found) TERM.{*lng*}. The search path is: user memory, system memory, then WRT:.

Under other versions of AMOS, AlphaWRITE doesn't know if your terminal is a 7-bit or 8-bit device. If {tdv}.{*lng*} file exists, AlphaWRITE knows you have a 7-bit device (if it was an 8-bit device, there wouldn't be a conversion table for it). If no conversion table exists, AlphaWRITE assumes the terminal is an 8-bit device.

The above applies to **display**, not input. You can only **directly input** 8-bit characters under AMOS 1.4 or 2.2, on an 8-bit device such as the AM-65 in native mode. However, you can **always** enter 8-bit characters with AlphaWRITE's compose-character

feature—regardless of your terminal's type or operating system level.

SUMMARY

AlphaWRITE 2.1 and later, allows users to enter via the keyboard and print via a laser printer the ISO Latin-1 character set. This support is available for *any* current version of AMOS and is not limited to AMOS 1.4, 2.2, and later.

8-bit display devices show the ISO Latin-1 characters directly on the screen. 7-bit display devices show equivalents as per terminal-language conversion tables. 8-bit characters are entered via hardware composition under AMOS 1.4, AMOS 2.2, or via software composition under any version of AMOS.

CHAPTER NINE
CONJUGATING VERB DICTIONARY

The purpose of the conjugating verb dictionary is to make AlphaWRITE's spell check feature usable by most Western European languages.

In Western European languages such as German, French, Spanish, and Italian, it is not practical to list every verb in a dictionary—even a printed dictionary. This is due to the many words that result from conjugating the verbs. In a printed dictionary, only the infinitive (base) form of verbs are listed, and the reader is expected to know the rules of conjugation.

AlphaWRITE allows spell checking to sense conjugated words in a document and look up the infinitive verb form in a special dictionary.

To use this verb conjugation feature, a message file table defining conjugation endings must be defined and a verb dictionary must be constructed.

Only regular verbs—verbs strictly adhering to the rules—may take advantage of this feature. Irregular verbs are in the main dictionary (SPELL.*lng* or SPELL.DCT).

The *AlphaWRITE Reference Guide*'s Chapter 15 - "AlphaWRITE's Dictionaries" provides more information on **SPELL**, the dictionaries it uses, and how to create a professional dictionary.

THE VERB DICTIONARY

AlphaWRITE automatically searches for and uses a dictionary named VERB.DCT. Like the other dictionaries, AlphaWRITE first looks for VERB.*lng*. If VERB.*lng* is not found, VERB.DCT is used. If no verb dictionary exists (the normal case for English), no error results, and the conjugation feature shuts down.

The location of the VERB dictionary is the same as for SPELL.DCT, the main dictionary. The location is typically WRT: unless the AlphaWRITE initialization file contains a DICTIONARY command to redirect AlphaWRITE to the new location for the dictionaries. DICTIONARY applies to SPELL.DCT and VERB.DCT.

VERB.DCT's internal structure is identical to that of a professional dictionary. The instructions for creating, sorting, and building a verb dictionary are the same as for a professional dictionary. The DICSRT and BLDPRF utilities are used. The VERB.DCT's creator must insure that only the infinitive forms of regular verbs are included in the verb dictionary.

DICTIONARY LOOK-UP SEQUENCE

AlphaWRITE consults dictionaries in this sequence:

1. The unrecognized word is looked for in SHORT.DCT.
2. The unrecognized word is looked for in SMALL.DCT.
3. The unrecognized word is looked for in SPELL.DCT.
4. If the unrecognized word ends in a conjugation ending, then the infinitive form of the word is looked for in VERB.DCT.
5. If a professional dictionary is defined, the unrecognized word is looked for in the professional dictionary.
6. The unrecognized word is looked for in LOCAL.DCT.

THE CONJUGATION TABLE

The WRT400.*lng* file contains a table that defines conjugation endings. In the source file (WRT400.M68) this is a table in the CMD: section, at label 7$:. Comments just before this label will confirm that this is the correct section.

For example, a conjugation table for German is shown below.

```
;conjugation table
;first string is standard ending
;other strings are conjugated endings
;table is terminated with a null byte

7$:       ASCIZ    "en"      ; infinitive ending (in VERB.DCT)

          ASCIZ    "e"       ; ich
          ASCIZ    "st"      ; du
          ASCIZ    "t"       ; er, sie, est, ihr
          ASCIZ    "en"      ; wir, sie
          BYTE     0         ; ** end of table **
          EVEN
```

The conjugation table is a series of null-terminated (ASCIZ) strings. The first string is the infinitive ending. The other strings are the conjugation endings. There may be any number of these. Finally, a byte of zero terminates the table.

If no conjugation is to take place (as in English), this table contains just a null byte as shown below.

```
7$:       BYTE     0         ; ** no conjugation **
```

APPENDIX A
PDVSYM.M68 AND PRINTER DRIVER SAMPLES

One method of creating a new printer driver is to copy and modify an existing printer driver file. You begin by choosing an existing printer driver whose characteristics most closely match those of the new printer. This appendix lists printer models and their corresponding printer driver file name.

Use the information in this appendix as a road map to find the printer driver source file accompanying the AlphaWRITE software.

Since AlphaWRITE's PDVSYM.M68 file and other printer driver source files frequently undergo change, the listings themselves are not reprinted here. However, you can print the .M68 files or call them up with AlphaVUE for reference when you are creating new printer drivers.

PDVSYM.M68 AND PRINTER DRIVER SOURCE FILE LOCATION

As of AlphaWRITE 2.1A and later, the AlphaWRITE software is downloaded into one account, WRT:. The PDVSYM.M68 file and all printer driver .M68 files can be found in WRT:, unless they have been relocated to MAC:.

Earlier versions of AlphaWRITE may have installed PDVSYM.M68 on DSK0:[7,7] and the printer driver .M68 files in DSK0:[7,12] (WRITE:) and/or DSK0:[10,2].

PRINTER DRIVER SOURCE ROAD MAP

PRINTER	DRIVER SOURCE
Dot Matrix types	MISC.M68
AM-301	AM301.M68
AM-302	AM302.M68
AM-306	AM306D.M68[1]
AM-306	AM306L.M68[2]
AM-306	AM306C.M68[3]
HP LaserJet I	LASJET.M68

PRINTER	DRIVER SOURCE
HP LaserJet Plus	LASJET.M68
HP LaserJet 500	LASJET.M68
HP LaserJet II	LASII.M68
HP LaserJet IID	LASIID.M68
HP LaserJet IIP	LASIID.M68
HP LaserJet III	LASIII.M68
HP LaserJet IIID	LASIII.M68
HP LaserJet IIIP	LASIII.M68
HP LaserJet IID/III/IIID installed with PostScript Cartridge	PSCART.M68
Apple LaserWriter	LASWRT.M68
Apple LaserWriter Plus	LASWRT.M68
Apple LaserWriter II	LASWRT.M68

[1] Draft quality
[2] Letter quality
[3] Correspondence quality

The LASWRT.M68 printer driver supports any printer Adobe Corp. has officially identified as a "PostScript Compatible" printer. PostScript printers other than Apple LaserWriters may or may not have the same fonts available. For example, while the NEC LC 890 has an identical font set to the Apple LaserWriter Plus, the IBM ProPrinter II has a different collection of fonts. Other laser printer drivers may be available; for more information call Alpha Microsystems Technical Support.

The MISC.M68 supports most dot matrix printers and supplies capability for some special print features.

APPENDIX B
THE ASCII AND ISO LATIN-1 CHARACTER SETS CHART

The following pages contain the ASCII and ISO Latin-1 codes for the characters to which they are assigned in octal, decimal, and hexadecimal. Codes 000 through 031 decimal are undisplayable control-characters; remaining codes represent printable characters unless described otherwise.

The chart is divided into two sections. The first section, codes 000 through 127 decimal, belongs to the ASCII character set. The first section, codes 000 trhough 127 decimal, plus the second section, codes 128 through 255 decimal, both belong to the ISO Latin-1 character set.

SYMBOL	OCTAL	DEC.	HEX.	MEANING
NULL	000	000	00	Null (fill character)
SOH	001	001	01	Start of Heading
STX	002	002	02	Start of Text
ETX	003	003	03	End of Text
ECT	004	004	04	End of Transmission
ENQ	005	005	05	Enquiry
ACK	006	006	06	Acknowledge
BEL	007	007	07	Bell code
BS	010	008	08	Back Space
HT	011	009	09	Horizontal Tab
LF	012	010	0A	Line Feed
VT	013	011	0B	Vertical Tab

SYMBOL	OCTAL	DEC.	HEX.	MEANING
FF	014	012	0C	Form Feed
CR	015	013	0D	Carriage Return
SO	016	014	0E	Shift Out
SI	017	015	0F	Shift In
DLE	020	016	10	Data Link Escape
DC1	021	017	11	Device Control 1
DC2	022	018	12	Device Control 2
DC3	023	019	13	Device Control 3
DC4	024	020	14	Device Control 4
NAK	025	021	15	Negative Acknowledge
SYN	026	022	16	Synchronous Idle
ETB	027	023	17	End of Transmission Blocks
CAN	030	024	18	Cancel
EM	031	025	19	End of Medium
SS	032	026	1A	Special Sequence
ESC	033	027	1B	Escape
FS	034	028	1C	File Separator
GS	035	029	1D	Group Separator
RS	036	030	1E	Record Separator
US	037	031	1F	Unit Separator
SP	040	032	20	Space
!	041	033	21	Exclamation Mark
"	042	034	22	Quotation Mark
#	043	035	23	Number Sign
$	044	036	24	Dollar Sign
%	045	037	25	Percent Sign
&	046	038	26	Ampersand
’	047	039	27	Apostrophe
(	050	040	28	Opening Parenthesis
)	051	041	29	Closing Parenthesis
*	052	042	2A	Asterisk
+	053	043	2B	Plus

SYMBOL	OCTAL	DEC.	HEX.	MEANING
,	054	044	2C	Comma
-	055	045	2D	Hyphen or Minus
.	056	046	2E	Period
/	057	047	2F	Slash
0	060	048	30	Zero
1	061	049	31	One
2	062	050	32	Two
3	063	051	33	Three
4	064	052	34	Four
5	065	053	35	Five
6	066	054	36	Six
7	067	055	37	Seven
8	070	056	38	Eight
9	071	057	39	Nine
:	072	058	3A	Colon
;	073	059	3B	Semicolon
<	074	060	3C	Left Angle Bracket
=	075	061	3D	Equal Sign
>	076	062	3E	Right Angle Bracket
?	077	063	3F	Question Mark
@	100	064	40	Commercial At
A	101	065	41	Upper Case Letter
B	102	066	42	Upper Case Letter
C	103	067	43	Upper Case Letter
D	104	068	44	Upper Case Letter
E	105	069	45	Upper Case Letter
F	106	070	46	Upper Case Letter
G	107	071	47	Upper Case Letter
H	110	072	48	Upper Case Letter
I	111	073	49	Upper Case Letter
J	112	074	4A	Upper Case Letter
K	113	075	4B	Upper Case Letter

SYMBOL	OCTAL	DEC.	HEX.	MEANING
L	114	076	4C	Upper Case Letter
M	115	077	4D	Upper Case Letter
N	116	078	4E	Upper Case Letter
O	117	079	4F	Upper Case Letter
P	120	080	50	Upper Case Letter
Q	121	081	51	Upper Case Letter
R	122	082	52	Upper Case Letter
S	123	083	53	Upper Case Letter
T	124	084	54	Upper Case Letter
U	125	085	55	Upper Case Letter
V	126	086	56	Upper Case Letter
W	127	087	57	Upper Case Letter
X	130	088	58	Upper Case Letter
Y	131	089	59	Upper Case Letter
Z	132	090	5A	Upper Case Letter
[	133	091	5B	Left Square Bracket
\	134	092	5C	Back Slash
]	135	093	5D	Right Square Bracket
^	136	094	5E	Circumflex
_	137	095	5F	Underline
‘	140	096	60	Grave Accent
a	141	097	61	Lower Case Letter
b	142	098	62	Lower Case Letter
c	143	099	63	Lower Case Letter
d	144	100	64	Lower Case Letter
e	145	101	65	Lower Case Letter
f	146	102	66	Lower Case Letter
g	147	103	67	Lower Case Letter
h	150	104	68	Lower Case Letter
i	151	105	69	Lower Case Letter
j	152	106	6A	Lower Case Letter
k	153	107	6B	Lower Case Letter

SYMBOL	OCTAL	DEC.	HEX.	MEANING
l	154	108	6C	Lower Case Letter
m	155	109	6D	Lower Case Letter
n	156	110	6E	Lower Case Letter
o	157	111	6F	Lower Case Letter
p	160	112	70	Lower Case Letter
q	161	113	71	Lower Case Letter
r	162	114	72	Lower Case Letter
s	163	115	73	Lower Case Letter
t	164	116	74	Lower Case Letter
u	165	117	75	Lower Case Letter
v	166	118	76	Lower Case Letter
w	167	119	77	Lower Case Letter
x	170	120	78	Lower Case Letter
y	171	121	79	Lower Case Letter
z	172	122	7A	Lower Case Letter
{{	173	123	7B	Left Brace
|	174	124	7C	Vertical Line
}	175	125	7D	Right Brace
~	176	126	7E	Tilde
	177	127	7F	Delete
ISO LATIN-1 CHARACTER SET				
	200-246	128-159	80-9F	Not assigned
	247	160	A0	Required space
¡	250	161	A1	Inverted exclamation
¢	251	162	A2	Cent sign
£	252	163	A3	Pound sterling
¨	253	164	A4	Currency
¥	254	165	A5	Yen
¦	255	166	A6	Split vertical bar
§	256	167	A7	Section symbol
È	257	168	A8	Umlaut or Dieresis

SYMBOL	OCTAL	DEC.	HEX.	MEANING
©	260	169	A9	Copyright symbol
ã	261	170	AA	Ord feminine
«	262	171	AB	Left guillemot
¬	263	172	AC	Not sign
-	264	173	AD	Discretionary hyphen
®	265	174	AE	Registered symbol
ð	266	175	AF	Hyphen or macron
°	267	176	B0	Ring or degree
±	270	177	B1	Plus or minus
²	271	178	B2	Raised 2
³	272	179	B3	Raised 3
Â	273	180	B4	Acute accent
µ	274	181	B5	Micro
¶	275	182	B6	Paragraph or pilcrow
•	276	183	B7	Raised dot
Ë	277	184	B8	Cedilla
¹	300	185	B9	Raised 1
ë	301	186	BA	Ord masculine
»	302	187	BB	Right guillemot
¼	303	188	BC	One quarter
½	304	189	BD	One half
¾	305	190	BE	Three quarters
¿	306	191	BF	Inverted question mark
À	307	192	C0	Capital A grave
Á	310	193	C1	Capital A acute
Â	311	194	C2	Capital A circumflex
Ã	312	195	C3	Capital A tilde
Ä	313	196	C4	Capital A umlaut
Å	314	197	C5	Capital A ring
Æ	315	198	C6	Capital AE ligature
Ç	316	199	C7	Capital C cedilla
È	317	200	C8	Capital E grave
É	320	201	C9	Capital E acute

SYMBOL	OCTAL	DEC.	HEX.	MEANING
Ê	321	202	CA	Capital E circumflex
Ë	322	203	CB	Capital E umlaut
Ì	323	204	CC	Capital I grave
Í	324	205	CD	Capital I acute
Î	325	206	CE	Capital I circumflex
Ï	326	207	CF	Capital I umlaut
ð	327	208	D0	Capital Eth
Ñ	330	209	D1	Capital N tilde
Ò	331	210	D2	Capital O grave
Ó	332	211	D3	Capital O acute
Ô	334	212	D4	Capital O circumflex
Õ	335	213	D5	Capital O tilde
Ö	336	214	D6	Capital O umlaut
?	337	215	D7	Multiplication symbol
Ø	330	216	D8	O slash
Ù	331	217	D9	Capital U grave
Ú	332	218	DA	Capital U acute
Û	333	219	DB	Capital U circumflex
Ü	334	220	DC	Capital U umlaut
?	335	221	DD	Capital Y acute
Þ	336	222	DE	Capital Thorn
ß	337	223	DF	Ess tset
à	340	224	E0	Lower case a grave
á	341	225	E1	Lower case a acute
â	342	226	E2	Lower case a circumflex
ã	343	227	E3	Lower case a tilde
ä	344	228	E4	Lower case a umlaut
å	345	229	E5	Lower case a ring
æ	346	230	E6	Lower case ae ligature
ç	347	231	E7	Lower case c cedilla
è	350	232	E8	Lower case e grave
é	351	233	E9	Lower case e acute
ê	352	234	EA	Lower case e circumflex

SYMBOL	OCTAL	DEC.	HEX.	MEANING
ë	353	235	EB	Lower case e umlaut
í	354	236	EC	Lower case i grave
í	355	237	ED	Lower case i acute
î	356	238	EE	Lower case i circumflex
ï	357	239	EF	Lower case i umlaut
ð	360	240	F0	Lower case eth
ñ	361	241	F1	Lower case n tilde
ò	362	242	F2	Lower case o grave
ó	363	243	F3	Lower case o acute
ô	364	244	F4	Lower case o circumflex
õ	365	245	F5	Lower case o tilde
ö	366	246	F6	Lower case o umlaut
?	367	247	F7	Division
ø	370	248	F8	Lower case o slash
ù	371	249	F9	Lower case u grave
ú	372	250	FA	Lower case u acute
û	373	251	FB	Lower case u circumlfex
ü	374	252	FC	Lower case u umlaut
?	375	253	FD	Lower case y acute
þ	376	254	FE	Lower case thorn
ÿ	377	255	FF	Lower case y umlaut

INDEX

/

/C option
  WRITE command 3-2
/F:filename option
  WRITE command 3-3
/NOD option
  WRITE command 3-3
/NOS option
  WRITE command 3-3
/P option
  WRITE command 3-2
/S option
  WRITE command 3-3

7

7-bit terminal 8-2, 8-3

8

8-bit characters 2-20, 8-1 to 8-3
8-bit terminal 8-2, 8-3

:

:/ symbol, comment character 3-6
:K option
  script files 3-6
:W option
  script files 3-6

A

AFM format
  font width table 2-39
AlphaWRITE
  message file process method 4-2, 4-3
AlphaWRITE 2.0
  printer drivers 2-7
AlphaWRITE 2.0 compatible
  printer driver flag 2-20
AlphaWRITE 2.1
  printer drivers 2-7
AlphaWRITE 2.1 compatible
  printer driver flag 2-20
AlphaWRITE Documentation Library 1-3
AlphaWRITE file
  reading with AWREAD XCALL 6-1
AlphaWRITE Installation Instructions 1-3
AlphaWRITE Keyboard Commands Reference 1-3
AlphaWRITE Reference Manual 1-3
AlphaWRITE Release Notes 1-3
AlphaWRITE versions
  printer driver compatibility 2-6
AM-65 terminal 8-2
AM301.PDV 2-2
AM302.PDV 2-2
AM306.PDV 2-2
AMOS 8-1, 8-3
Apostrophe
  command line processing 3-4
Applications
  printer drivers for 2-7
ASCII character set 2-20
  chart B-1 to B-8
  extended characters 5-1
ASCII statement 4-7, 4-9
ASCIZ statement 4-9
Assembler directives 4-3, 4-4, 4-15
Assembling message files 4-16
AWREAD.SBR XCALL 6-1

B

BYTE statement 4-8

C

Capability flags 2-4, 2-15
Caret symbol
  command line processing 3-4
Cartridges supporting fonts 2-38
Change bars
  P.CBAR 2-36
Character composition 8-1 to 8-3
  hardware 8-2
  software 8-2
Character set
  ASCII 2-20
  charts B-1 to B-8
  ISO Latin-1 2-20
  translation tables 8-3
Characters
  outputting 2-10
  substitution during print 8-3
Characters per inch 2-39
Close (shutdown) printer function 2-21
Colon, script file character 3-7
Command area 4-12
Command dispatch area 4-13
Command line processing 3-1 to 3-9
Command table area 4-13
Commas
  command line processing 3-3, 3-4
Comment character 3-6
Communication area 2-8
Compatibility 2-6
Conjugating verb dictionary 9-1
Conjugation table
  VERB.DCT 9-2
Conventions 1-1
  used in this book 1-1
CRT statement 4-7
Cut sheet feeder flag 2-16
Cut-sheet feeders
  P.CLS routine 2-21

D

Definition table
  extended characters 5-1
Device-dependent print effect
  P.DDPE 2-37
Dictionary
  VERB.DCT 9-1, 9-2
Direct move to coordinate location 2-31
Directly output byte 2-31
Disable proportional spacing function 2-24
Display attributes
  AlphaWRITE's handling of 4-7
Documentation 1-3
  overview of this book 1-1
Double underscore function 2-27

E

Enable proportional spacing function 2-23
Entry Points 2-2
Error handling 3-4
Examples
  printer driver 2-11
Extended characters 5-1, 5-2
  scope of in AlphaWRITE 5-3
Extended printer drivers 2-5, A-1
  flag 2-16
  standards 2-1 to 2-41

F

Field area 4-14
Field dispatch area 4-14
  translating 4-15
Field table area 4-14
Field terminals 4-7
File names
  message files 4-1, 4-2
Filename option
  WRITE command 3-2
FIXTRN utility 5-2
Flags longword 2-39
FO.CAR
  cartridges supporting fonts 2-38
FO.CPI
  characters per inch 2-39
FO.FLG
  font flags 2-38
FO.LED
  leading in points 2-38
FO.LOG
  logical name of font 2-37
FO.PHY
  physical name of font 2-38
FO.SIZ
  font size in points 2-38
FO.TBL
  name of font width table 2-38
Font flags 2-38
Font size in points 2-38
Font table 2-37
Font table structure 2-37
Font width tables 2-39, 2-40
Functions
  printer driver 2-21

G

Graphics functions flag 2-17
Gray printing flag 2-17

H

Halting
  command line processing 3-4
Header 4-4
Help files 4-16
HP Laser printing
  ISO Latin-1 Characters 8-2

I

Image processing
  off function 2-35
  on function 2-34
Implementation 2-6
Impure area 2-8
Initialization 2-10
Initialize printer function 2-21
ISO Latin-1 character set 8-1 to 8-3
  chart B-1 to B-8
  printer driver flag 2-20
  transformation to NRCS 7-1 to 7-4
Italics flag 2-17

K

Kerning
  pair 2-40
Keyboard
  mapping extended characters 5-2
Keycap printing flag 2-20

L

Label 4-5
Language definition files
  international characters 8-3
LASJET.PDV 2-1
LASWRT.PDV 2-1
Leading control flag 2-18
Leading in points 2-38
Lines
  outputting 2-10
Logical name of font 2-37

M

M68 Assembler 4-16
Measurement
  extended printer drivers 2-5
  recommendations 2-6
Message area
  translating 4-10
Message dispatch area 4-5
Message files
  assembled name 4-16
  assembling 4-16
  how AlphaWRITE processes 4-2, 4-3
  overview 4-2
  translating 4-1, 4-2, 4-6, 4-7
Message table area 4-5, 4-6
Messages 4-5
MISC.PDV 2-2
Mode terminals 4-7
Move to character position 2-30
Move to left paper offset function 2-26
Move to line position in d2. 2-31
Move to next line function 2-23
Move to top margin function 2-25

N

Name of font width table 7Font table 2-38
NRCS characters
  equivalents to ISO Latin-1 8-2
  transformation from ISO Latin-1 7-1 to 7-4

O

Offsets 4-4
Operating systems
  character composition 8-1, 8-2
Organization
  of this book 1-1
Outline fonts 2-36
Output character function 2-21
Output extended characters function 2-28, 2-34
Output multiple line feeds function 2-27
Output page overlay function 2-34
Output special character function 2-33
Output special control string function 2-23
Overprint function 2-26

P

P.BAR
  toggle overbar 2-24
P.BOLD
  toggle bold 2-24
P.BYTE
  4directly output byte 2-31
P.CART
  set cartridge/printwheel/thimble 2-35
P.CBAR
  toggle change bars 2-36
P.CHR
  Output character 2-21
P.CLS
  close (shutdown) printer 2-21
P.CRLF
  move to next line 2-23
P.CTL
  perform control function 2-22
P.DBL
  toggle double underscore 2-27
P.DDPE
  device-dependent print effect 2-37
P.ECHR
  output extended characters 2-28
P.FATR
  set font attributes directly 2-30
P.FNAM
  set font by name 2-29
P.FONT
  select font family 2-27
P.FSIZ
  set font size 2-29
P.FSUB
  perform font substitution 2-33
P.FTBL
  return index to font table 2-31
P.GRAF
  perform graphics function 2-32
P.GRAY
  set gray shading 2-32
P.IMOF
  image processing off 2-35
P.IMON
  image processing on 2-34
P.INI
  initialize printer 2-21
P.ITAL
  toggle italicized printing 2-29
P.KERN
  set track kerning 2-35
P.KEY
  toggle keycap printing 2-35
P.LEAD
  set leading 2-29
P.LEFT
  set left margin in points 2-28
P.LF
  output multiple line feeds 2-27
P.MCHR
  move to character position 2-30
P.MLIN
  move to line position in d2. 2-31
P.MLPO
  move to left paper offset 2-26
P.MOVE
  direct move to coordinate location 2-31
P.MTM
  move to top margin 2-25
P.OTEC
  output extended character 2-34
P.OTSC
  output special character 2-33
P.OUTL
  toggle outline fonts 2-36
P.OVRP
  overprint 2-26
P.PCH
  return printer characteristics 2-36
P.POVR
  output page overlay 2-34
P.PSIZ
  set page size in points 2-28
P.PSOF
  disable proportional spacing 2-24
P.PSON
  enable proportional spacing 2-23
P.REST
  restore print state 2-31
P.REV
  toggle reversed printing 2-30
P.SAVE
  save print state 2-31
P.SCPI
  set characters per inch (pitch) 2-25
P.SCRN
  toggle screened printing 2-30
P.SETL
  set line weight 2-34
P.SHMI
  set horizontal motion index 2-25
P.SLPI
  set vertical motion index 2-24
P.SLPO
  set left paper offset 2-25
P.SLPP
  set lines per page function 2-26
P.SLSP
  set line spacing 2-26
P.SPAG
  set page printing 2-36
P.SPL
  output special control string 2-23
P.SPOR
  set page orientation 2-32
P.STM
  set top margin 2-25
P.STRK
  toggle strikeout 2-24
P.TMAR
  set top margin in points 2-28
P.TOF
  move to top of form 2-23

P.TREC
translate extended character 2-33
P.TRSC
translate special character 2-33
P.UNDR
toggle underscore 2-24
Page printing
P.SPAG 2-36
Pages
outputting 2-10
Pair kerning 2-40
PD$BLD
printer supports bold 2-17
PD$EDG
printer can't print at edge of page 2-19
PD$EXT
extended printer driver 2-16
PD$FED
cut sheet feeder 2-16
PD$FNT
printer has selectable fonts 2-18
PD$GRF
printer supports graphics functions 2-17
PD$GRY
printer supports gray printing 2-17
PD$IMG
printer can process images 2-19
PD$ISO
printer driver flag 2-20
PD$ITL
printer supports italics 2-17
PD$KEY
printer supports keycap printing 2-20
PD$KRN
printer supports track kerning 2-20
PD$LED
printer has leading control 2-18
PD$PRO
proportional spacing 2-16
PD$PTS
printer driver is points-oriented 2-16
PD$REV
printer supports reversed printing 2-17
PD$SAV
save/restore print state 2-16
PD$SCR
printer supports screened printing 2-17
PD$SFC
selectable font cartridges 2-19
PD$TOP
top-of-character orientation 2-19
PD$UND
printer supports underscore 2-17
PD$VAR
variable spacing 2-15
PD$W20
printer driver flag 2-20
PD$W20 flag
printer driver version 2-7
PD$W21
printer driver flag 2-20
PD$W21 flag
printer driver version 2-7
PD$WH1
supports type 1 printwheels 2-18
PD$WH2
supports type 2 printwheels 2-18
PDVSYM.M68 2-2
Perform control function 2-22
Perform font substitution function 2-33
Perform graphics function 2-32
Physical name of font 2-38
Points-oriented printer driver flag 2-16
POS statement 4-7
PostScript laser printing
ISO Latin-1 Characters 8-2
PostScript printer driver A-2
Printer chracteristics
P.PCH 2-36
Printer definitions
PD$W20 and PD$W21 flags 2-7
Printer drivers A-2
application requirements of 2-7
communication area 2-8
compatibility between versions 2-6
examples of creating 2-11
extended 2-1 to 2-40
impure area 2-8
register calling 2-9
samples A-1
structure 2-2
version compatiblity 2-6
version number 2-7
Printer supports bold flag 2-17
Printers 8-3
Printing
8-bit characters 8-2, 8-3
Process images flag 2-19
Program Header 2-4
Proportional fonts 2-14
Proportional spacing 2-13
Proportional spacing flag 2-16
PSCART.PDV 2-1

R

Recommendations 2-6
Registers
how to call 2-9
Restore print state 2-31
Return index to font table 2-31
Reversed printing flag 2-17
Right justify 2-12

S

Save print state 2-31
Save/restore print state flag 2-16
Screened printing flag 2-17
Script file, colon as character 3-7
Script files 3-1 to 3-9
chaining 3-6
Select font family function 2-27
Selectable font cartridges flag 2-19
Selectable fonts flag 2-18
Set cartridge/printwheel/thimble 2-35
Set characters per inch (pitch) 2-25
Set font attributes directly 2-30
Set font by name 2-29
Set font size 2-29
Set gray shading 2-32
Set horizontal motion index function 2-25
Set leading 2-29
Set left margin in points 2-28
Set left paper offset function 2-25
Set line spacing function 2-26
Set line weight function 2-34
Set lines per page function 2-26
Set page orientation 2-32
Set page size in points 2-28
Set top margin function 2-25
Set top margin in points 2-28
Set track kerning function 2-35
Set vertical motion index function 2-24
Source code
printers drivers A-1

Spell checking
  conjugating verb dictionary 9-1
SPLNEG
  spooling default 2-21
SPLPOS
  spooling default 2-21
Standards
  extended printer drivers 2-1 to 2-41

T

Terminals
  character composition 8-1, 8-2
  mode and field 4-7
Toggle bold function 2-24
Toggle italicized printing 2-29
Toggle keycap printing function 2-35
Toggle overbar function 2-24
Toggle reversed printing 2-30
Toggle screened printing 2-30
Toggle strikeout function 2-24
Toggle underscore function 2-24
Top of form (start next page) 2-23
Top-of-character orientation flag 2-19
Track kerning 2-15
Track kerning flag 2-20
Transformation tables 7-1 to 7-4
Translate extended character function 2-33
Translate special character function 2-33
Translation tables
  character set conversion 8-3
Type 1 printwheels flag 2-18
Type 2 printwheels flag 2-18
Type-ahead feature
  command line processing 3-4

U

Underscore flag 2-17

V

Variable spacing flag 2-15
Verb dictionary 9-1, 9-2

W

WRITE command
  command line format 3-2
  command line processing 3-1 to 3-9
WRT400.lng
  verb conjugation table 9-2

X

XCALL
  AWREAD.SBR 6-1